

Министерство цифрового развития, связи и массовых коммуникаций Россий-
ской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Е. Ю. Мерзлякова

**Визуальное программирование и человеко-машинное
взаимодействие**

Учебно-методическое пособие

Новосибирск 2025

УДК 004.5(075.8)

Утверждено редакционно-издательским советом СибГУТИ

Рецензент к.т.н. зав. каф.ТСиВС В.Г. Дроздова

Мерзлякова Е. Ю. Визуальное программирование и человеко-машинное взаимодействие/ Е. Ю. Мерзлякова; Сибирский государственный университет телекоммуникаций и информатики; каф. прикладной математики и кибернетики. – Новосибирск, 2025. – 76 с.

Учебно-методическое пособие предназначено для студентов технических специальностей, изучающих дисциплину «Визуальное программирование и человеко-машинное взаимодействие» в соответствии с программой подготовки бакалавров по направлению 09.03.01 «Информатика и вычислительная техника» профиля «Программное обеспечение средств вычислительной техники и автоматизированных систем». Пособие содержит краткие теоретические сведения по созданию приложений в среде Qt Creator на языке C++, а также методические указания к выполнению практических заданий и курсовой работы.

© Мерзлякова Е.Ю., 2025

© Сибирский государственный университет
телекоммуникаций и информатики, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1. РАЗРАБОТКА ПРИЛОЖЕНИЙ В IDE QT CREATOR.....	5
1.1 Пользовательский интерфейс Qt Creator.....	5
1.2 Создание проекта.....	6
1.3 Qt Designer.....	13
1.4 Структура и функциональность библиотеки Qt.....	16
1.5 Механизм сигналов и слотов.....	18
1.6 Работа с виджетами.....	23
1.7 Архитектура модель/представление.....	25
1.8 Создание меню приложения.....	28
1.9 Создание диалоговых окон	32
1.10 Контрольные вопросы.....	39
2. МЕТОДИЧЕСКИЕ УКАЗАНИЯ ПО ВЫПОЛНЕНИЮ ПРАКТИЧЕСКИХ РАБОТ В IDE QT CREATOR	40
2.1 Практическая работа №1.....	40
2.2 Практическая работа №2.....	47
2.3 Практическая работа №3.....	53
2.4 Практическая работа №4.....	55
2.5 Практическая работа №5.....	57
2.6 Практическая работа №6.....	62
2.7 Практическая работа №7.....	66
3. МЕТОДИЧЕСКИЕ УКАЗАНИЯ К КУРСОВОЙ РАБОТЕ.....	69
3.1 Задание.....	69
3.2 Требования к отчету курсовой работы.....	81
3.3 Контрольные вопросы для защиты курсовой работы.....	84
СПИСОК ЛИТЕРАТУРЫ.....	85

ВВЕДЕНИЕ

Индустрия программного обеспечения в России сегодня развивается высокими темпами. Прикладное ПО – наиболее распространенная категория, применяемая в самых разных сферах человеческой деятельности. К программным средствам общего назначения относятся текстовые и графические редакторы, игры, электронные таблицы, СУБД, а также комплексные системы с широким функционалом. Одним из популярных инструментов для разработки таких программ является Qt Creator – мощная кросс-платформенная IDE для проектирования приложений с современными и гибкими интерфейсами.

Библиотека Qt, разработанная в 1996 году на C++, остается востребованной среди разработчиков для создания десктопных и мобильных решений, ПО для микрокомпьютеров, промышленной автоматизации, транспорта и медицинского оборудования. Сегодня Qt применяют для высокопроизводительных приложений: мессенджеров, игр, ресурсоемких программ, а также в отраслях с повышенными требованиями к безопасности.

Qt лицензируется LGPLv3 и GPLv2, а инструменты Qt для разработчиков, такие как Qt Creator и qmake, а также некоторые модули поставляются под лицензией GPLv3 (Open Source) – самой известной из свободных лицензий. Данная лицензия требует, чтобы исходные коды разработанных программ были также открытыми под GPLv3.

Данное учебно-методическое пособие содержит руководство по работе в интегрированной среде разработки Qt Creator 4.8.0 (основанном на Qt 5.12.0), а также описание библиотеки Qt и возможности ее применения в разработке прикладного ПО. Предполагается, что читатель уже знаком с программированием на C++ и имеет представление об объектно-ориентированном программировании. В пособии также содержатся подробные рекомендации по выполнению практических заданий с объяснением основных моментов. Пошаговое выполнение представленных практических работ поможет освоить процесс разработки приложений в среде Qt Creator от простого к сложному. В данное пособие включены также методические указания к выполнению курсовой работы в ходе которой обучающиеся смогут провести цикл разработки приложения по предложенным вариантам.

1. ПРОГРАММИРОВАНИЕ В IDE QT CREATOR

1.1 Пользовательский интерфейс Qt Creator

При запуске QtCreator, открывается режим «Начало» (рис.1.1.1), в котором можно открыть последние проекты или создать новые, посмотреть примеры проектов, открыть учебники в виде текстовых и видеоматериалов на английском языке. Также в данном режиме можно перейти на официальный сайт Qt Group [1] к своей учетной записи, форуму или открыть справку.

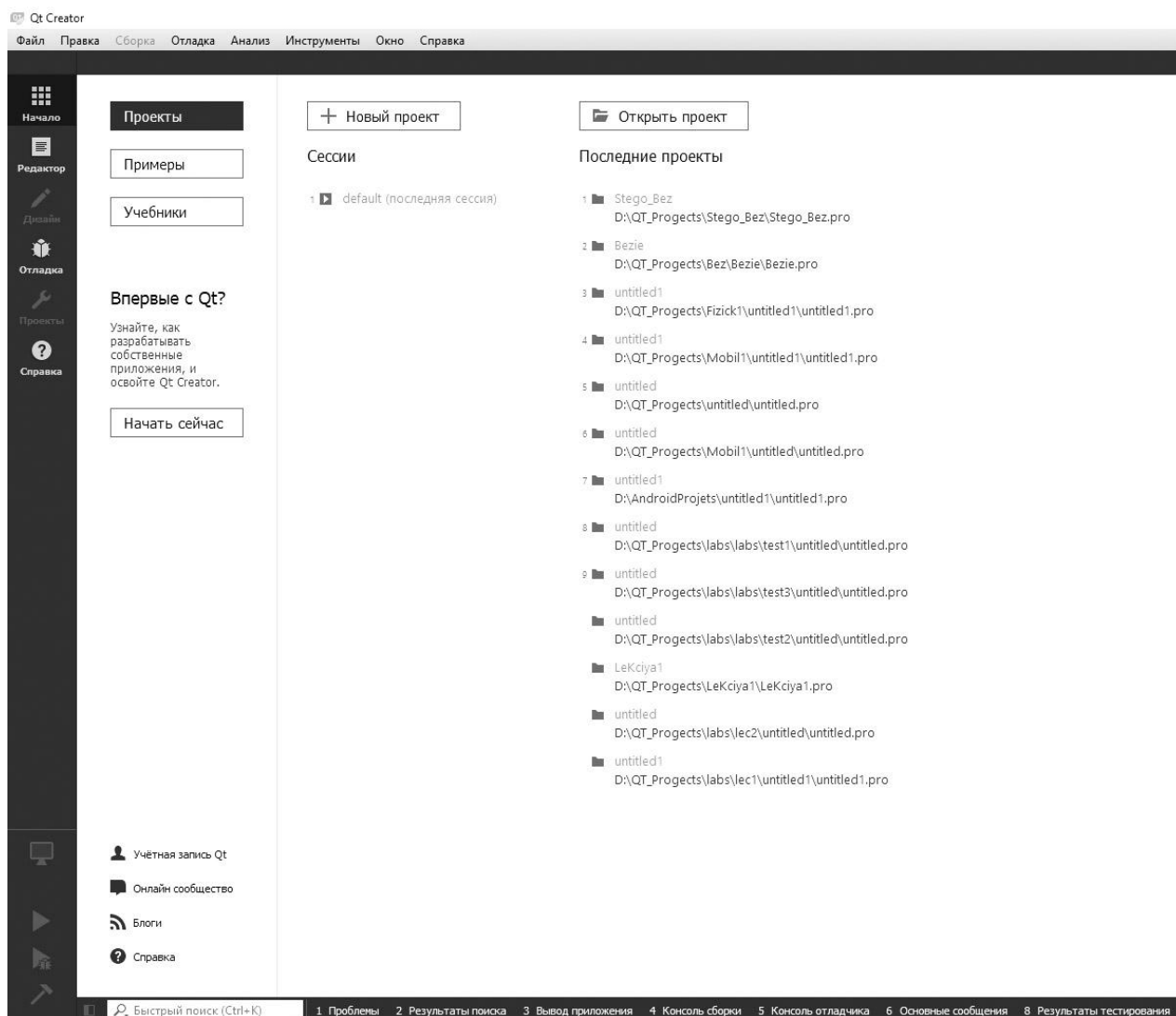


Рисунок 1.1.1 – Режим «Начало» в QtCreator

С помощью панели слева вы можете переходить в другие режимы:

- «Редактор» - для редактирования проекта и файлов исходных программных кодов;
- «Дизайн» - для создания пользовательских интерфейсов приложений;
- «Отладка» - для просмотра состояния программы при отладке;
- «Проекты» - для управления текущими проектами;

- «Справка» - содержит Qt Creator Manual.

Нижняя часть панели содержит столбец со значками переключения для запуска, отладки и сборки проекта. Эти значки активируются при работе над проектом.

В верхней части главного окна находится строка меню, пункты которого автоматически активируются или деактивируются в зависимости от текущего состояния приложения.

В нижней части главного окна доступна панель задач, содержащая режимы вывода:

- «Проблемы» - предоставляет список проблем, таких, как сообщения об ошибке или предупреждения, которые нужно исправить. Здесь отфильтровываются незначительные сообщения компилятора и упорядочивается его вывод;
- «Результаты поиска» - показывает результаты поисков в текущем документе, в проекте или в файлах на диске;
- «Вывод приложения» - отображает статус программы при ее выполнении и отладочную информацию;
- «Консоль сборки» - выводит весь вывод компилятора;
- «Консоль отладчика» - отображает исходную информацию отладчика в явном и сжатом виде;
- «Основные сообщения» - выводит главные сообщения;
- «Контроль версий» - показывает сообщения от системы контроля версий;
- «Результаты тестирования» - выводит результаты запущенных тестов.

1.2 Создание проекта

Чтобы создать новый проект, нужно либо нажать на кнопку «Новый проект» в режиме «Начало», либо выбрать в меню «Файл» опцию «Создать файл или проект...». После этого откроется окно для создания нового проекта (рис.1.2.1), в котором слева необходимо выбрать шаблон «Приложение», а справа параметр «Приложение Qt Widgets». Это нужно при создании приложения Qt для настольных компьютеров. На следующем шаге необходимо указать название и путь размещения проекта (рис.1.2.2), затем выбрать конфигурацию компилятора (рис.1.2.3). Стандартным для Linux и Mac OS X является компилятор GCC. Для Windows можно воспользоваться его свободным аналогом – MinGW или компилятором MSVC, который входит в состав Microsoft Windows SDK или Visual Studio. После выбора комплекта, потребуется ввести информацию о классах, для которых будут автоматически созданы шаблоны исходных текстов. Для главного окна программы будет сгенерирован класс MainWindow, который унаследован от класса QMainWindow. Данный класс имеет все свойства, характерные для главного окна программы [2]. Также для исходного кода класса MainWindow автоматически будут созданы заголовочный файл mainwin-

dow.h и файл реализации mainwindow.cpp. Флаг «Создать форму» указывает на то, что будет автоматически создана и добавлена к проекту форма главного окна (рис.1.2.4).

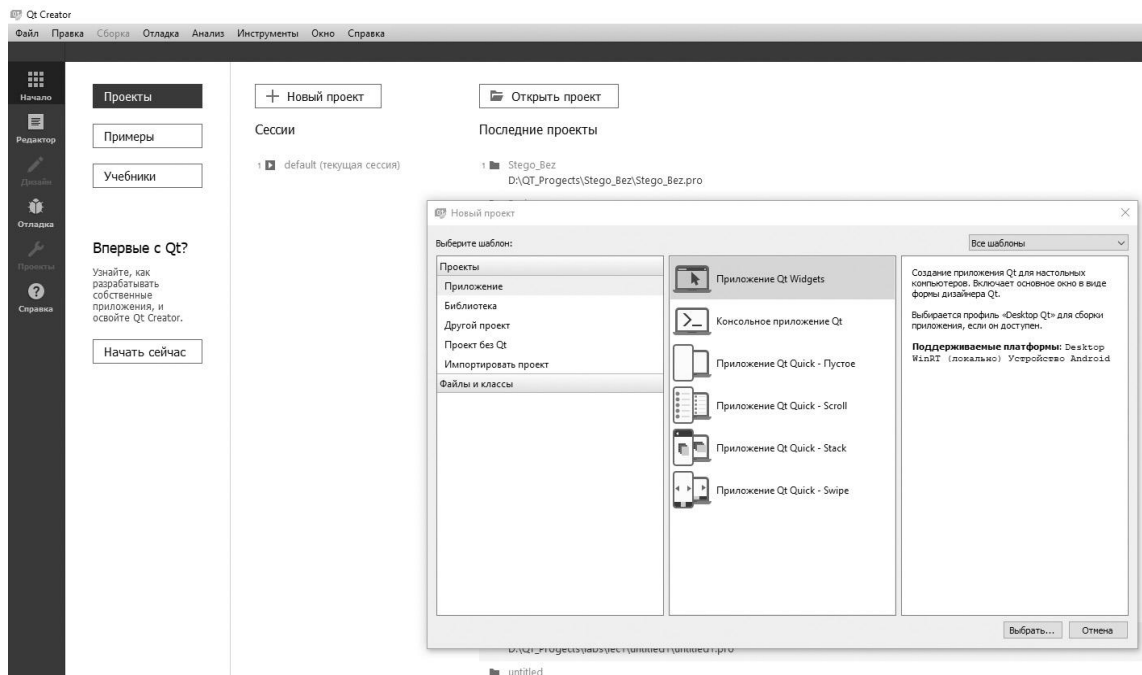


Рисунок 1.2.1 – Окно создания проекта

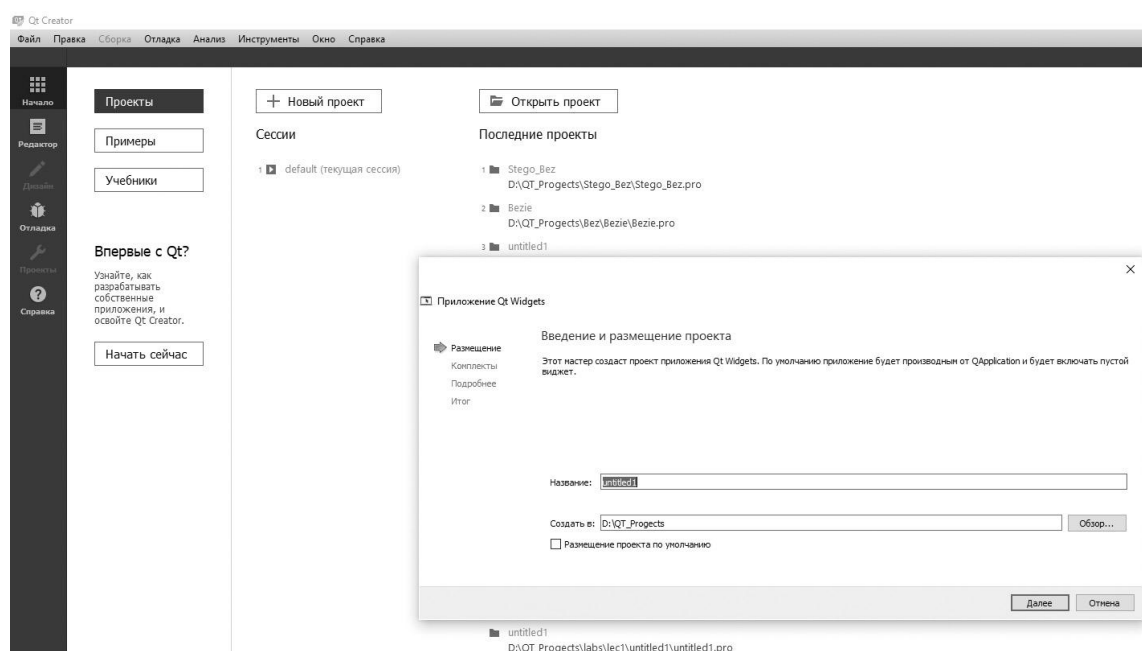


Рисунок 1.2.2 – Выбор пути размещения проекта

После перечисленных действий, нужно нажать на кнопку «Завершить» и в результате будет создан проект, который откроется в окне QtCreator. В левой части окна будет отображена структура проекта (рис. 1.2.5), где находится

главный файл проекта с расширением .pro, описывающий входящие в проект файлы и настройки программы.

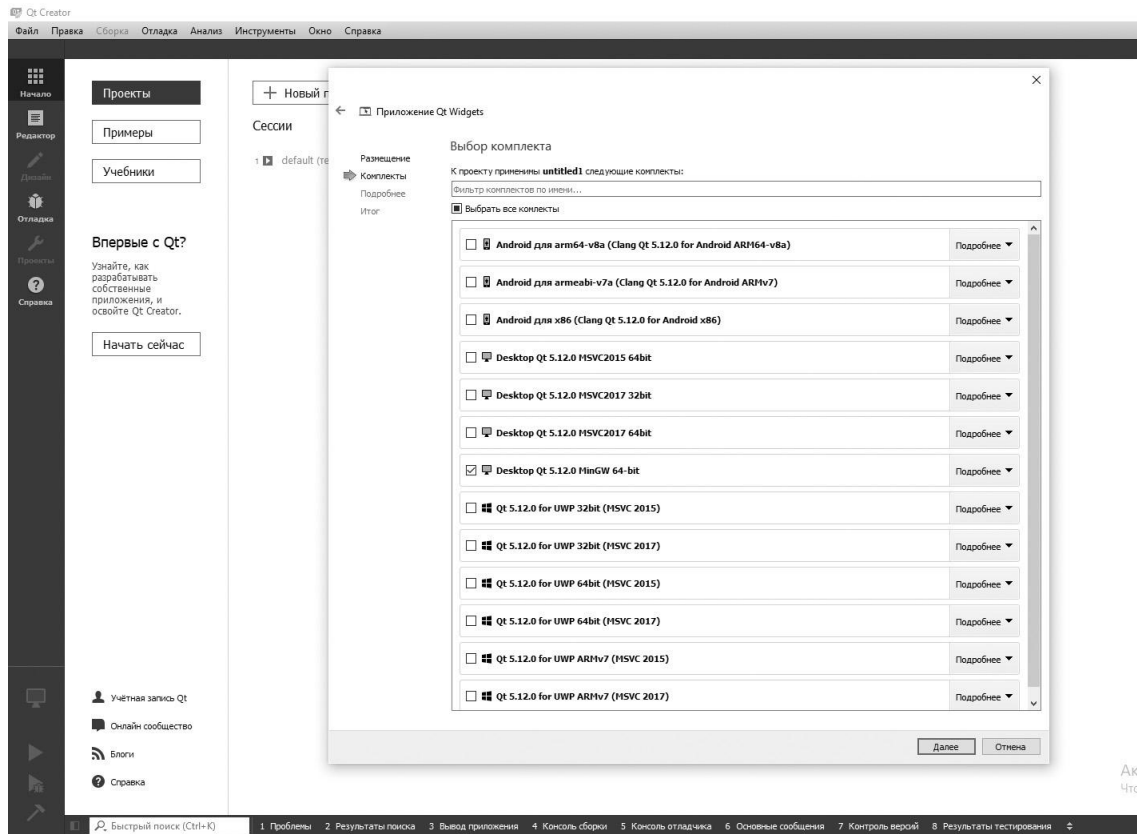


Рисунок 1.2.3 – Выбор конфигурации

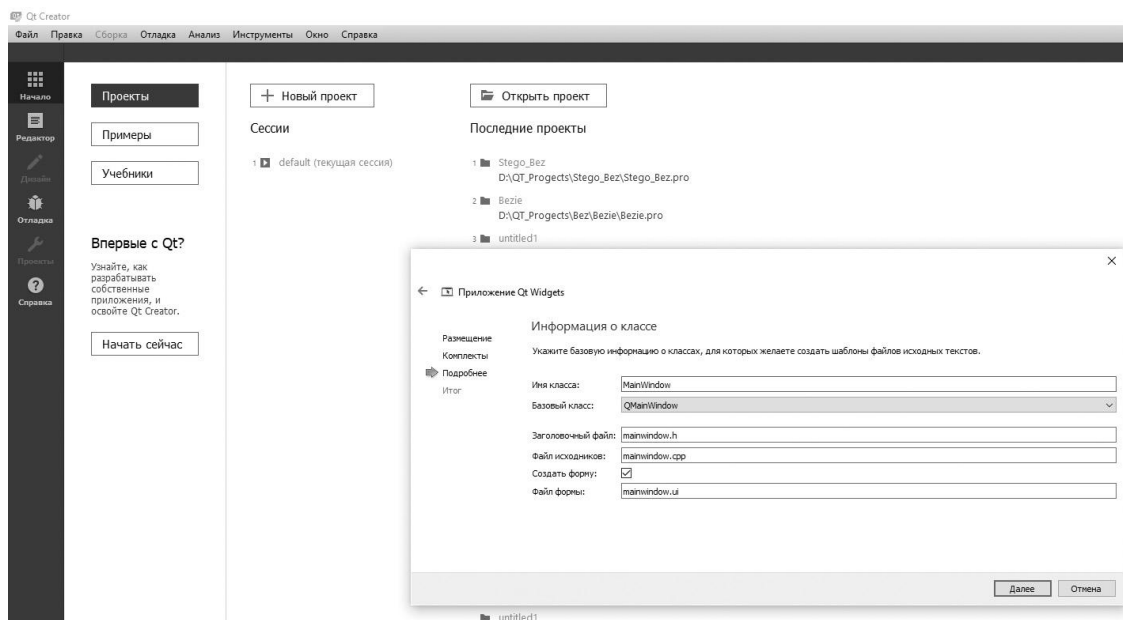


Рисунок 1.2.4 – Информация о классе

Рассмотрим подробнее основные переменные настроек проекта:

- HEADERS – заголовочные файлы проекта с расширением .h;

- CONFIG – параметры компилятора, настройки конфигурации проекта с помощью следующих флагов:
 - release – построение проекта в режиме релиза;
 - debug – построение проекта в режиме отладки;
 - debug_and_release – проект готов для сборки в режимах отладки и выпуска;
 - debug_and_release_target – параметр по умолчанию;
 - build_all – при установленном флаге debug_and_release проект будет создан в режимах отладки и выпуска;
 - autogen_precompile_source – установка автоматической генерации файла .cpp, включающего предварительно скомпилированный файл заголовка, указанный в файле .pro;
 - ordered – при использовании флага subdirs, этот параметр указывает, что перечисленные каталоги должны обрабатываться в том порядке, в котором они указаны;
 - precompile_header – включает поддержку для использования precompiled headers в проектах;
 - warn_on – при установке этого флага компилятор должен выводить как можно больше предупреждений;
 - warn_off – при установке этого флага компилятор должен выводить как можно меньше предупреждений;
 - exceptions – поддержка исключений, включена по умолчанию;
 - exceptions_off – отключена поддержка исключений;
 - rtti – включение поддержки rtti;
 - rtti_off – отключение поддержки rtti;
 - stl – включена поддержка stl;
 - stl_off – отключена поддержка stl;
 - thread – включена поддержка потоков;
 - c++11 – включена поддержка C++11;
 - c++14 – включена поддержка C++14;
- QT – используемые в проекте модули Qt. По умолчанию указано ядро и графический интерфейс;
- TARGET – название исполняемого файла;
- TEMPLATE – тип проекта: app, lib, subdirs, aux, vcapp или vclib;
- SOURCES – все исходные файлы проекта с расширением .cpp;
- FORMS – файлы, описывающие формы Qt Designer;
- DEFINES – макроопределения в проекте, аналогичные директиве пре-процессора #define;
- RESOURCES – указывает на файл ресурсов;
- VERSION – версия проекта.

Пример простого проекта:

```

1 #-----
2 #
3 # Project created by QtCreator 2023-04-12T19:21:51
4 #
5 #-----
6
7 QT      += core gui
8
9 greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
10
11 TARGET = BCES
12 TEMPLATE = app
13 DEFINES += QT_DEPRECATED_WARNINGS
14
15 CONFIG += c++11
16
17 SOURCES += \
18     main.cpp \
19     mainwindow.cpp
20
21 HEADERS += \
22     mainwindow.h
23
24 FORMS += \
25     mainwindow.ui
26
27 # Default rules for deployment.
28 qnx: target.path = /tmp/${TARGET}/bin
29 else: unix:!android: target.path = /opt/${TARGET}/bin
30 !isEmpty(target.path): INSTALLS += target
31 |

```

В строке 7 с помощью переменной QT автоматически подключились модули библиотеки Qt – Core и Gui. В строке 9 подключается также модуль Widgets в том случае, если основная версия Qt (QT_MAJOR_VERSION) является 4. В строке 11 указано название исполняемого файла. В строке 12 указан тип проекта – приложение. В строке 15 задан параметр компилятора, который указывает на использование C++11. В строках с 17 по 19 перечислены все исходные файлы проекта. Строки 21-22 указывают на заголовочный файл проекта, а строки 24-25 – на файл формы. В строках 28-30 определены некоторые настройки по умолчанию, которые QtCreator предоставляет для развертывания данного приложения на разных платформах:

- в строке 28 указан путь установки (target.path) для систем QNX, /tmp/\${TARGET}/bin, где \${TARGET} будет заменен на имя приложения;
- в строке 29 задан путь установки /opt/\${TARGET}/bin для систем UNIX, исключая Android (unix:!android:);

- строка 30 проверяет, не является ли target.path пустым, и если это так, добавляет название проекта из TARGET в список установки.

Также проект содержит заголовочный файл, файлы исходных кодов и файл формы, выбрав который, откроется режим дизайна (рис. 1.2.6).

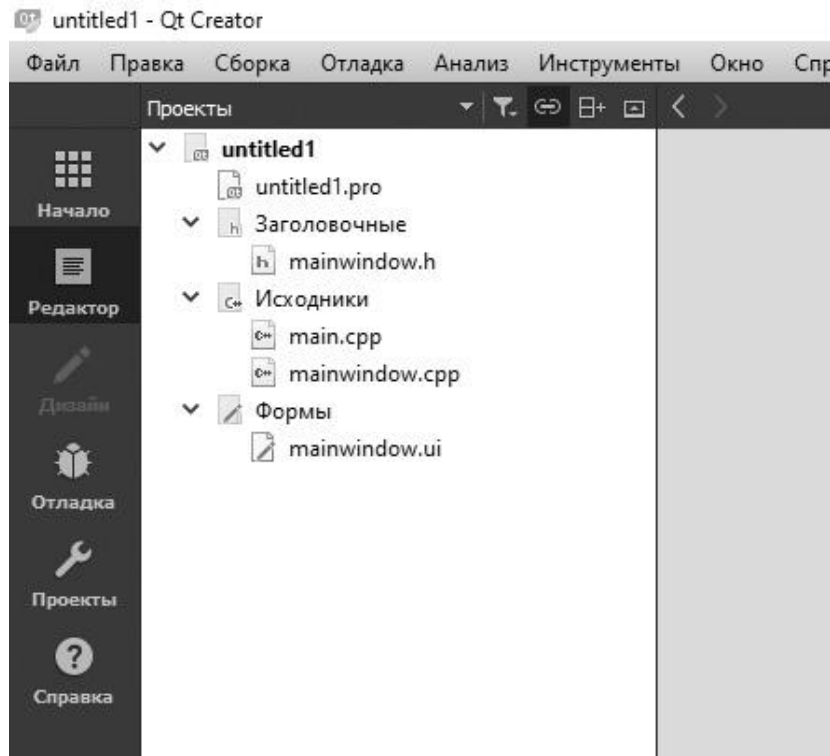


Рисунок 1.2.5 – Структура проекта

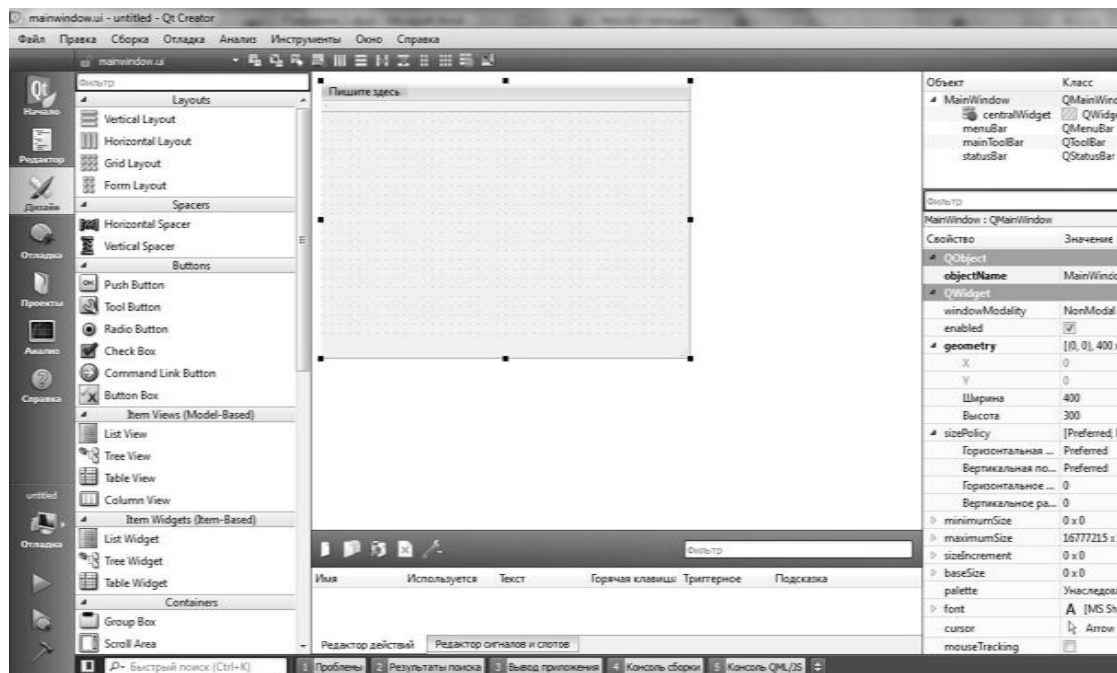


Рисунок 1.2.6 – Форма mainwindow.ui

Созданный шаблон можно скомпилировать и запустить, нажав на панели слева кнопку запуска, после чего появится пустое окно – главное окно программы (рис.1.2.7).

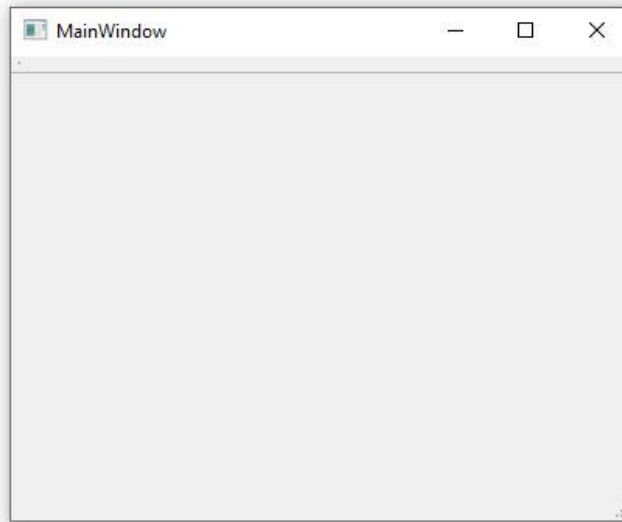


Рисунок 1.2.7 – Главное окно приложения

Все файлы проекта были автоматически сгенерированы, но можно и создавать их вручную. Рассмотрим содержание файла main.cpp (рис. 1.2.8):

```
1  #include "mainwindow.h"
2  #include <QApplication>
3
4  int main(int argc, char *argv[])
5  {
6      QApplication a(argc, argv);
7      MainWindow w;
8      w.show();
9
10     return a.exec();
11 }
```

Рисунок 1.2.8 – Исходный код main.cpp

В данном файле создается главное окно приложения, которое мы рассмотрели. В первых строках в программу подключается определение класса `QApplication` и заголовочный файл для `mainwindow`. В строке 6 создается экземпляр класса `QApplication`, который управляет ресурсами приложения. Конструктору `QApplication` передаются те же аргументы, что и у функции `main`, так как Qt может обрабатывать аргументы командной строки. Далее, в строке 7 создается главное окно приложения, которое является экземпляром класса `QMainWindow`. В строке 8 окно выводится на экран, а в строке 10 выполняется передача управ-

ления библиотеке Qt. После этого программа переходит в режим ожидания, во время которого она только ждет действий от пользователя.

Рассмотрим содержание файла `mainwindow.cpp` (рис.1.2.9):



```
1  #include "mainwindow.h"
2  #include "ui_mainwindow.h"
3
4  MainWindow::MainWindow(QWidget *parent) :
5      QMainWindow(parent),
6      ui(new Ui::MainWindow)
7  {
8      ui->setupUi(this);
9  }
10
11 MainWindow::~MainWindow()
12 {
13     delete ui;
14 }
```

Рисунок 1.2.9 – Исходный код `mainwindow.cpp`

В данном файле находится конструктор и деструктор класса `MainWindow`, который предоставляет все необходимое для разработки оконного приложения (меню, панели инструментов, панель информации и т.д.).

1.3 Qt Designer

Программа Qt Designer – это инструмент для проектирования и создания графических пользовательских интерфейсов. Он предоставляет возможность быстро создавать прототипы приложений, которые базируются на диалоговых окнах, а также могут иметь главное окно, меню, строку состояния и панель инструментов. Работать с Qt Designer можно в Qt Creator в режиме «Дизайн».

Исходные элементы для создания пользовательского интерфейса в Qt называются виджетами. Виджеты и формы, созданные с помощью Qt Designer, легко интегрируются с запрограммированным кодом, и все свойства, установленные Qt Designer, могут быть динамически изменены в коде.

Рассмотрим интерфейс Qt Designer (рис.1.3.1). Главное окно содержит меню и панели инструментов. В левой части окна находится панель с виджетами, где можно найти необходимые виджеты по категориям:

- Layouts – виджеты компоновки;
- Spacers – разделители пространства;
- Buttons – кнопки и переключатели;
- Items View (Model-Based) – представления для элементов данных на основе моделей;

- Items Widgets (Item-Based) - представления для элементов данных на основе записей;
- Containers – виджеты для групп объектов;
- Input Widgets – виджеты для ввода данных;
- Display Widgets – виджеты для демонстрации информации;

Виджеты можно добавлять на форму с помощью перетаскивания. Список используемых на форме виджетов содержится в окне инспектора объектов, расположенного справа вверху окна Qt Designer. В данном окне можно выбирать виджеты для их изменения.

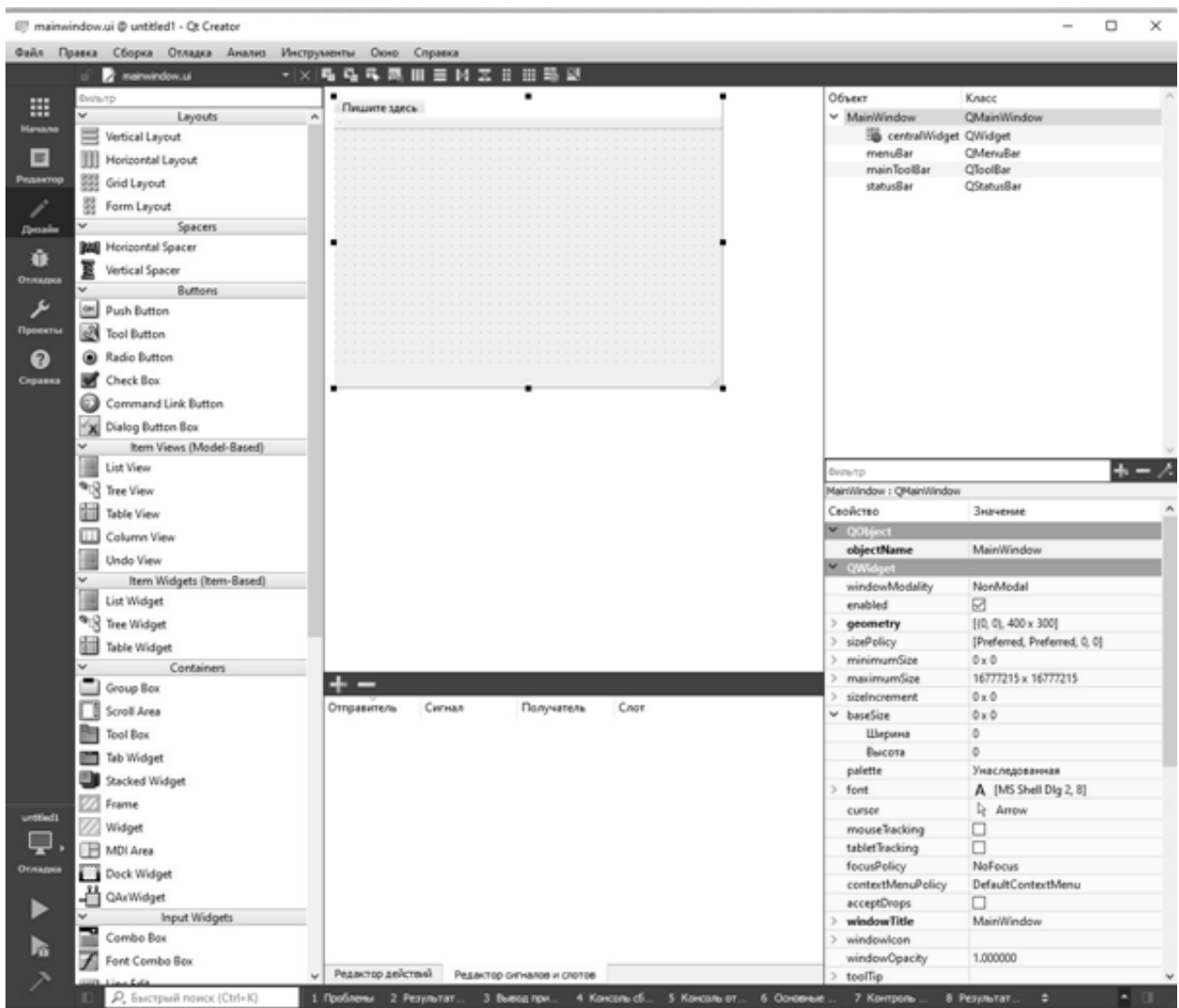


Рисунок 1.3.1 – Qt Designer

Справа снизу располагается окно редактора свойств, определяющего свойства выбранного виджета. Данное окно содержит таблицу свойств и их значений. Например, выберем форму MainWindow и найдем свойство «windowTitle» в редакторе свойств. Вписав любое другое значение, например, «Программа», мы увидим результат на самой форме в запущенном приложении (рис. 1.3.2):

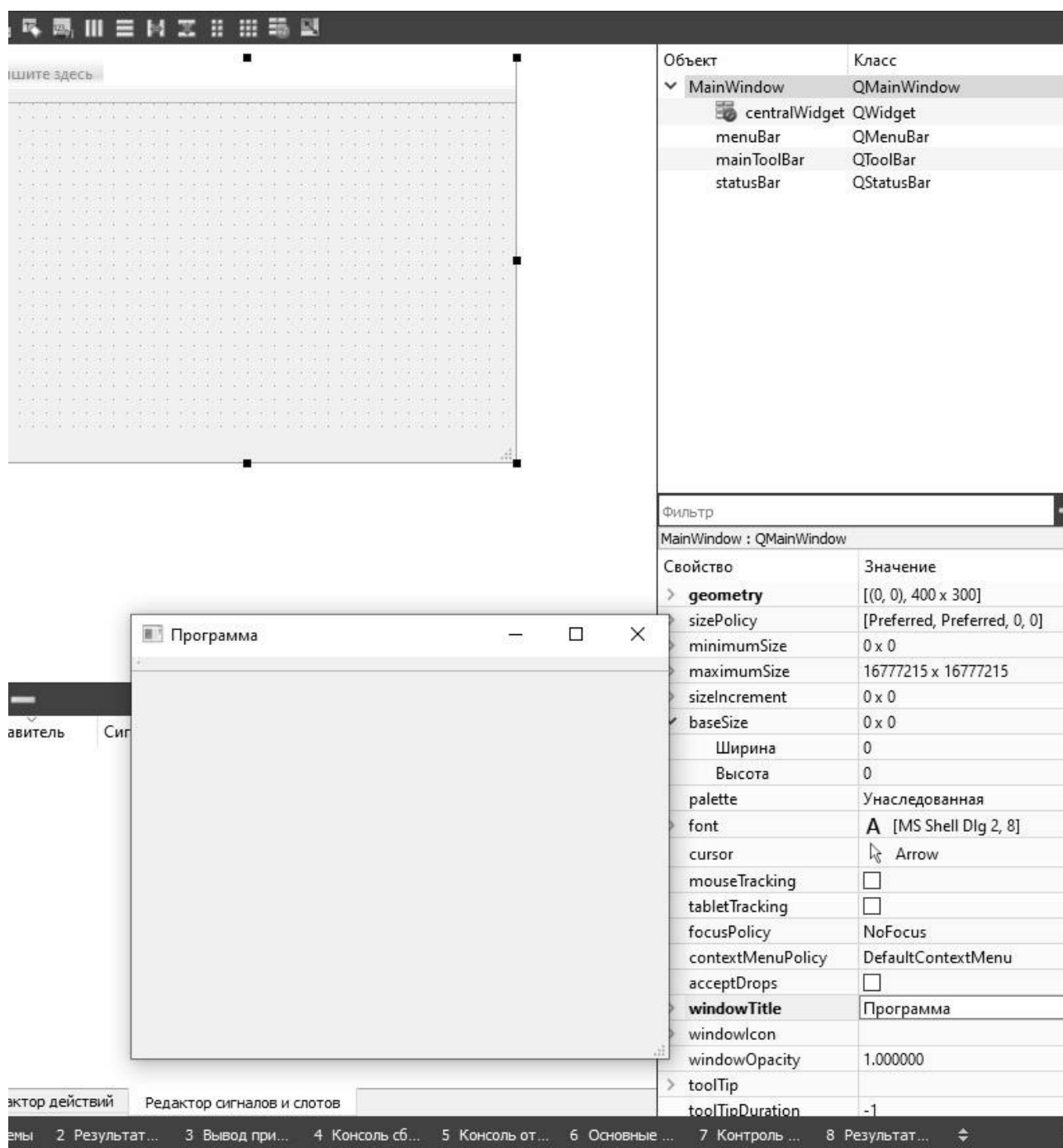


Рисунок 1.3.2 – Свойства формы

Вернемся к Qt Designer. В нижней части главного окна Qt Creator расположены вспомогательные панели, включая окна редактора сигналов и слотов и редактора действий. Редактор сигналов и слотов служит для редактирования соединений сигналов со слотами. Сигналы представляют собой уведомления о происходящих изменениях или действиях, а слоты представляют собой методы, которые присоединяются к сигналам. Окно редактора действий предназначено для создания, управления и удаления действий команд создаваемой формы.

1.4 Структура и функциональность библиотеки Qt

Библиотека Qt 5.12.0 предоставляет более 2000 классов, организованных в модули, для разработки кросс-платформенных приложений. В Qt Creator 4.8.0, основанном на этой версии, реализована расширенная поддержка данных модулей, включая улучшенный анализ кода и отладку. Все программы, имеющие графический интерфейс, используют модули QtCore, QtGui и QtWidgets. Модуль QtCore является фундаментальной составляющей платформы Qt и обеспечивает базовую функциональность, необходимую для построения как GUI-, так и консольных приложений. Среди ключевых компонентов модуля выделяются:

- Система метаобъектов (Meta-Object System) , реализующая механизм сигналов и слотов (с использованием макроса Q_OBJECT), что позволяет создавать гибкую межобъектную коммуникацию.
- Контейнерные классы, такие как QList<T>, QVector<T>, QMap<K, V>, QHash<K, V>, предоставляющие типизированные коллекции данных с удобным и безопасным доступом.
- Классы для работы с вводом-выводом, включая QFile, QTextStream, QDataStream, которые обеспечивают работу с файлами, потоками данных, буферизацией и поддержкой различных кодировок.
- Подсистема обработки событий, представленная классами QEvent и QApplication, реализующая цикл событий (event loop) — основу реактивности приложения.
- Средства многопоточности, такие как QThread, QThreadPool, QMutex, предоставляющие платформонезависимый интерфейс для параллельного выполнения задач и синхронизации потоков.

Модуль QtGui предоставляет базовые функции для отображения графики и взаимодействия с оконной системой. Он обеспечивает абстракцию над графическими подсистемами операционных систем, позволяя разработчикам создавать визуальные элементы интерфейса независимо от используемой платформы. Основные возможности модуля включают:

- Рендеринг 2D-графики с помощью класса QPainter, поддерживающего аппаратное ускорение через такие технологии, как OpenGL и Raster.
- Работу с изображениями: QImage — предназначен для прямого доступа к пиксельным данным и обработки изображений в памяти; QPixmap — оптимизирован для быстрого отображения графики на экране.
- Интеграцию с OpenGL через классы QOpenGLContext и QOpenGLFunctions, что позволяет использовать современные возможности GPU для создания высокопроизводительной графики.
- Управление шрифтами и текстом, реализованное с помощью классов QFont (задание параметров шрифта) и QFontMetrics (определение размеров текста при отображении).
- Обработку событий низкого уровня, связанных с вводом (мышь, клавиатура), а также управление окнами и поверхностями отображения.

Модуль QtGui является ключевым компонентом для построения графических приложений, обеспечивая низкоуровневое взаимодействие с графической подсистемой ОС и являясь основой для более высокоуровневых модулей, таких как QtWidgets и QtQuick .

Модуль QtWidgets предоставляет набор классических компонентов для разработки графических интерфейсов в десктопных приложениях. Он построен на базе модулей QtCore и QtGui , обеспечивая удобные высокоуровневые элементы управления и механизмы их организации. Основные возможности модуля:

- Базовый класс виджетов QWidget , являющийся основой всех пользовательских интерфейсных элементов. Он реализует обработку событий, отрисовку и взаимодействие с пользователем.
- Система компоновки интерфейса, основанная на классах QLayout и его производных (QHBoxLayout, QVBoxLayout, QGridLayout), которые обеспечивают автоматическое расположение элементов с адаптацией к изменению размеров окна.
- Стандартные элементы управления, такие как кнопки (QPushButton), поля ввода (QLineEdit), списки (QComboBox), таблицы (QTableView) и другие, позволяющие создавать полноценные интерфейсы без необходимости разработки компонентов с нуля.
- Поддержка диалоговых окон: классы QDialog, QMessageBox, QFileDialog предоставляют готовые решения для взаимодействия с пользователем, включая выбор файлов, отображение сообщений и модальные формы ввода данных.

Кроме базовых модулей, фреймворк Qt включает ряд специализированных библиотек, расширяющих его функциональные возможности:

- QtQuick, QtQML – для быстрой разработки графических интерфейсов на базе QML, JavaScript и возможностей Qt;
- QtNetwork – предоставляет средства сетевого взаимодействия, включая работу с TCP- и UDP-сокетами, а также поддержку клиентских приложений, использующих протоколы HTTP и FTP;
- QtXml, QtXmlPatterns – содержат классы для обработки XML-документов, включая парсинг, сериализацию и применение XPath/XSLT-запросов;
- QSql – обеспечивает доступ к реляционным базам данных через SQL, поддерживая различные драйверы и механизмы выполнения запросов;
- QtMultimedia, QtMultimediaWidgets – предоставляют функционал для разработки приложений с поддержкой аудио-, видеоданных и других мультимедийных возможностей;
- QtSvg – модуль поддержки графического векторного формата SVG, базирующегося на XML.

Перечисленные модули представляют собой основные компоненты фреймворка Qt 5.12.0 , охватывающие широкий спектр задач разработки приложений: от создания графических интерфейсов до работы с сетью, базами данных и

мультимедиа. Помимо описанных модулей, Qt содержит множество других специализированных библиотек. Например:

- QtWebEngine - для встраивания веб-браузера в приложение.
- QtBluetooth - для работы с Bluetooth-устройствами.
- QtSerialPort - для взаимодействия с COM-портами.
- QtPositioning - для работы с GPS и геолокацией.
- Qt3D - для создания 3D-графики.

Эти и некоторые другие модули выходят за рамки данного пособия, так как относятся к более узкоспециализированным областям разработки.

1.5 Механизм сигналов и слотов

В процессе разработки приложений с графическим интерфейсом часто возникает необходимость реагировать на различные действия пользователя и на соответствующие изменения состояния элементов интерфейса. Для организации таких взаимодействий в Qt реализован механизм сигналов и слотов, который позволяет обмениваться информацией о событиях, вырабатываемых объектами и реагировать на них соответствующим образом. При этом объекты не зависят друг от друга напрямую, что делает простым для расширения.

Каждый класс, наследуемый от QObject, может принимать и отправлять сигналы. Сигнал (signal) — это уведомление, которое отправляется объектом при наступлении определённого события. Например, при нажатии на кнопку вызывается сигнал clicked(), при изменении текста в поле ввода — сигнал textChanged() и т. д. Слот (slot) — это метод, который может быть вызван автоматически при получении сигнала. Слоты могут быть обычными функциями, статическими методами, а также лямбда-выражениями. Сигнал можно соединять с разным количеством слотов, а слот может принимать сообщения от многих сигналов, принадлежащих разным объектам.

Для связи между сигналом и слотом используется статический метод connect() из класса QObject. В общем виде, вызов метода connect() выглядит следующим образом [3]:

```
bool QObject::connect(
    const QObject *sender, //источник события
    const char *signal, //сигнал
    const QObject *receiver, //объект-приемник
    const char *method, //функция-обработчик
    Qt::ConnectionType type = Qt::AutoConnection //режим обработки
) const;
```

Для указания сигнала и слота ранее использовались макросы SIGNAL() и SLOT(), но начиная с Qt 5 рекомендуется использовать типобезопасную запись: connect(sender, &Sender::signalName, receiver, &Receiver::slotName);

Такой подход позволяет избежать ошибок на этапе компиляции и обеспечивает лучшую читаемость кода. Рассмотрим простой пример использования механизма сигналов и слотов:

```
#include <QApplication>
#include <QPushButton>
#include <QLabel>
#include <QVBoxLayout>
#include <QWidget>

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);

    QWidget window;
    QVBoxLayout layout(&window);

    QPushButton button("Показать/скрыть");
    QLabel label("Скрытый текст");

    // Связываем сигнал clicked со стандартным методом setVisible у QLabel
    QObject::connect(&button, &QPushButton::clicked, &label,
&QLabel::setVisible);

    layout.addWidget(&button);
    layout.addWidget(&label);
    window.resize(300, 100);
    window.show();

    return app.exec();
}
```

В данном примере при каждом нажатии на кнопку в приложении генерируется сигнал `clicked()`, который автоматически передаёт значение `true` или `false`. Это значение указывает, активирован ли элемент в данный момент. Получив такой сигнал, метка (`QLabel`) вызывает свой встроенный слот `setVisible()`, который отвечает за отображение или скрытие элемента на экране. Таким образом, при каждом клике по кнопке состояние метки меняется: если она была видна — исчезает, а если была скрыта — снова появляется.

В библиотеке Qt большинство элементов интерфейса, такие как кнопки, поля ввода и списки, уже имеют стандартные сигналы и слоты, предназначенные для обработки типовых событий. Например, сигнал `clicked()` у кнопки или `textChanged()` у текстового поля позволяют отслеживать действия пользователя. Слоты, в свою очередь, реализуют реакцию на эти события — например, закрытие окна или обновление данных на форме. Использование стандартных сигналов и слотов упрощает разработку, но также можно создавать и свои сигналы и слоты в пользовательских классах. Рассмотрим реализацию собственного сигнала и слота на примере счётчика, который генерирует сигнал при изменении своего значения. В данном примере реализуется пользовательский класс `Counter`, предназначенный для отслеживания и изменения числового значения с

возможностью уведомления других объектов об этом изменении. Класс наследуется от `QObject`, что позволяет использовать механизм сигналов и слотов:

Заголовочный файл `counter.h`:

```
#ifndef COUNTER_H
#define COUNTER_H

#include <QObject>

class Counter : public QObject {
    Q_OBJECT

public:
    explicit Counter(QObject *parent = nullptr);

signals:
    void valueChanged(int newValue); // Сигнал изменения значения

public slots:
    void increment(); // Слот увеличения счётчика
};

#endif // COUNTER_H
```

В заголовочном файле `counter.h` объявляется интерфейс класса: конструктор принимает указатель на родительский объект типа `QObject` и объявляется сигнал `valueChanged(int newValue)`, который будет вызываться при каждом изменении внутреннего значения счётчика. Также объявлен слот `increment()`, который увеличивает значение счётчика.

Файл `main.cpp`:

```
#include <QCoreApplication>
#include "counter.h"

int main(int argc, char *argv[]) {
    QCoreApplication app(argc, argv);
    Counter counter;
    // Подписываемся на сигнал valueChanged
    QObject::connect(&counter, &Counter::valueChanged, [](int val) {
        if (val >= 5) {
            qDebug() << "Достигнут порог!";
            exit(0);
        }
    });

    QTimer::singleShot(1000, &counter, &Counter::increment);
    QTimer::singleShot(2000, &counter, &Counter::increment);
    QTimer::singleShot(3000, &counter, &Counter::increment);
    QTimer::singleShot(4000, &counter, &Counter::increment);
    QTimer::singleShot(5000, &counter, &Counter::increment);

    return app.exec();
}
```

В файле `main.cpp` реализуется консольное приложение, в котором создается объект класса `Counter`, и каждую секунду вызывается его метод `increment()`, увеличивающий внутренний счётчик. При каждом изменении значения генерируется сигнал `valueChanged(int)`, на который подписано лямбда-выражение. Оно проверяет, достигло ли значение счётчика числа 5, и если да — выводит сообщение и завершает программу.

Файл `counter.cpp`:

```
#include "counter.h"
#include <QDebug>

Counter::Counter(QObject *parent) : QObject(parent) {}

void Counter::increment() {
    static int count = 0;
    count++;
    qDebug() << "Текущее значение:" << count;
    emit valueChanged(count); // Вызываем сигнал
}
```

Здесь каждый раз при вызове метода `increment()` после обновления значения счетчика вызывается сигнал `valueChanged(count)`, уведомляя все подписанные на него объекты о произошедшем изменении. Таким образом, класс `Counter` предоставляет функционал простого счётчика, который может не только хранить и изменять своё состояние, но и информировать другие части программы о каждом изменении с помощью механизма сигналов и слотов.

Механизм сигналов и слотов в Qt позволяет не только связывать один сигнал с одним слотом, но и подключать к одному сигналу сразу несколько слотов. Это означает, что при возникновении определённого события (например, нажатия кнопки или изменения значения в виджете) можно выполнить несколько действий независимо друг от друга. Рассмотрим пример, в котором при каждом нажатии на кнопку одновременно обновляется счётчик и появляется информационное окно:

```
#include <QApplication>
#include <QPushButton>
#include <QLabel>
#include <QVBoxLayout>
#include <QWidget>
#include <QMessageBox>

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);

    QWidget window;
    QVBoxLayout layout(&window);

    QPushButton button("Нажми меня");
    QLabel label("Счётчик: 0");

    int count = 0;
```

```

// Первый слот — обновление метки
QObject::connect(&button, &QPushButton::clicked, [&label, &count]() {
    count++;
    label.setText(QString("Счётчик: %1").arg(count));
});

// Второй слот — вывод сообщения
QObject::connect(&button, &QPushButton::clicked, []() {
    QMessageBox::information(nullptr, "Сообщение", "Кнопка нажата!");
});

layout.addWidget(&button);
layout.addWidget(&label);

window.show();
return app.exec();
}

```

В данном примере основное окно содержит кнопку и текстовую метку, которая отображает текущее значение счётчика. При нажатии на кнопку происходит два действия: значение счётчика увеличивается на единицу и обновляется надпись на экране, а также выводится всплывающее окно с сообщением о том, что кнопка была нажата. Таким образом, один и тот же сигнал `clicked()` кнопки связан сразу с двумя разными слотами.

Механизм сигналов и слотов в Qt позволяет не только связывать объекты между собой, но и гибко управлять этой связью. Для настройки взаимодействия между отправителем сигнала и его получателем Qt предоставляет несколько режимов подключения, задаваемых параметром `Qt::ConnectionType`, каждый из них влияет на порядок и условия выполнения слота:

- `Qt::AutoConnection` — автоматический выбор режима соединения в зависимости от того, в каком потоке находятся источник и получатель сигнала. Если они находятся в одном потоке, используется прямой вызов; если в разных — через очередь событий.
- `Qt::DirectConnection` — слот вызывается сразу после отправки сигнала, в том же потоке, где находится его источник. Вызов происходит напрямую, без задержек.
- `Qt::QueuedConnection` — слот выполняется не сразу, а добавляется в очередь событий. Фактически он будет вызван позже, когда управление вернётся к циклу обработки событий. Используется, когда объекты находятся в разных потоках.
- `Qt::BlockingQueuedConnection` — аналогичен `Qt::QueuedConnection`, но вызывающий поток блокируется до тех пор, пока слот не будет выполнен. Особенно полезен при необходимости синхронной передачи данных между потоками.

- `Qt::UniqueConnection` — гарантирует, что соединение между данным сигналом и слотом будет создано только один раз. Если такое соединение уже существует, повторный вызов `connect()` будет проигнорирован.

Эти типы соединений позволяют гибко управлять поведением взаимодействия между объектами, особенно в многопоточных приложениях, обеспечивая стабильность и предсказуемость работы программы.

Если необходимость в обработке сигнала отпадает, можно вручную разорвать связь с помощью метода `disconnect()`. Однако в большинстве случаев это не требуется, так как Qt автоматически очищает все соединения при удалении объектов.

1.6 Работа с виджетами

Виджет — это элемент графического интерфейса, который может отображать информацию, принимать ввод от пользователя или служить контейнером для других виджетов. Qt представляет собой мощную библиотеку виджетов, предназначенную для разработки современных графических приложений. Все виджеты наследуются от класса `QWidget`, который предоставляет общую функциональность: обработку событий, рисование, компоновку и взаимодействие с родительскими/дочерними элементами.

Все виджеты, то есть элементы интерфейса, можно условно разделить на несколько логических групп, каждая из которых выполняет определённые функции и обеспечивает удобное взаимодействие с пользователем. Базовые элементы управления, через которые пользователь взаимодействует с приложением — это кнопки (`QPushButton`, `QToolButton`), переключатели (`QRadioButton`), флажки (`QCheckBox`), а также элементы выбора значений: ползунки (`QSlider`), числовые поля (`QSpinBox`, `QDoubleSpinBox`), выпадающие списки (`QComboBox`) и специализированные редакторы дат и времени (`QDateEdit`, `QTimeEdit`). Эти элементы обеспечивают выполнение действий, выбор состояний и задание параметров. Например, стандартная кнопка `QPushButton` может выполнять какое-либо действие при нажатии:

```
QPushButton *button = new QPushButton("Нажми меня", &window);
QObject::connect(button, &QPushButton::clicked, &app, &QApplication::quit);
```

Если требуется выбрать один вариант из нескольких, используется переключатель `QRadioButton`. Группировка таких элементов позволяет организовать выбор только одного варианта в рамках набора. Для независимого выбора состояния применяется `QCheckBox`. Элементы вроде `QSlider`, `QSpinBox` или `QComboBox` служат для задания числовых значений или выбора из списка. Например, ползунок позволяет изменять значение в диапазоне от 0 до 100:

```
QSlider *slider = new QSlider(Qt::Horizontal);
slider->setRange(0, 100);
connect(slider, &QSlider::valueChanged, this, &SliderWindow::onSliderMoved);
```

Для работы с текстом используются текстовые компоненты: статичные метки (QLabel), цифровые табло (QLCDNumber), однострочные и многострочные поля ввода (QLineEdit, QTextEdit, QPlainTextEdit), а также просмотрщики форматированного текста (QTextBrowser). Они позволяют как отображать, так и редактировать текстовую информацию, поддерживая разный уровень сложности — от простого ввода до полноценного форматирования. Пример обработки ввода:

```
QLineEdit *lineEdit = new QLineEdit;
connect(lineEdit, &QLineEdit::returnPressed, [lineEdit]() {
    qDebug() << "Введено:" << lineEdit->text();
});
```

Более сложные элементы, такие как QTextEdit и QPlainTextEdit, поддерживают многострочный ввод и форматирование текста, что делает их подходящими для редакторов документов.

Интерфейс можно гибко структурировать и организовывать с помощью организационных контейнеров. Например, группа с заголовком (QGroupBox) визуально объединяет элементы внутри рамки. С помощью QTabWidget можно создать интерфейс с вкладками, где каждая вкладка содержит свой контент. Для постраничного отображения подходит QStackedWidget, а динамическое расположение окон реализуется через QMdiArea. Док-панели (QDockWidget) позволяют строить гибкие рабочие пространства, как в полноценных IDE.

Для отображения структурированных данных предусмотрены специализированные виджеты: списки (QListWidget), деревья (QTreeWidget), таблицы (QTableWidget), колоночные представления (QColumnView), календари (QCalendarWidget) и графические холсты (QGraphicsView). Они подходят для отображения информации в различных форматах, начиная от простых записей и заканчивая сложными иерархическими и графическими данными. Например, список (QListWidget) удобен для последовательного вывода элементов, дерево (QTreeWidget) — для иерархического представления информации, а таблица (QTableWidget) — для табличной организации данных. Вот как можно добавить элементы в список:

```
QListWidget *listWidget = new QListWidget;
listWidget->addItems({"Элемент 1", "Элемент 2", "Элемент 3"});
```

Взаимодействие с пользователем может происходить и через стандартные диалоговые окна, такие как выбор файла (QFileDialog), выбор цвета (QColorDialog), настройка шрифтов (QFontDialog), запрос значения (QInputDialog), вывод сообщений (QMessageBox) и индикаторы прогресса (QProgressDialog). Эти окна предоставляют готовые решения для типовых задач и обеспечивают единообразие интерфейса. Далее мы будем останавливаться на них подробнее.

Для построения сложных приложений используется структура главного окна, реализованная через QMainWindow и его компоненты: меню (QMenuBar), па-

нели инструментов (QToolBar), строку состояния (QStatusBar), а также док-панели (QDockWidget), что позволяет создавать профессиональные и масштабируемые интерфейсы.

Существует и ряд специализированных виджетов, ориентированных на конкретные задачи: воспроизведение видео (QVideoWidget, QMediaPlayer), вывод с камеры (QCameraViewfinder), отображение векторной графики (QSvgWidget), 3D-рендеринг (QOpenGLWidget, QRhiWidget), интеграция с веб-технологиями (QWebEngineView) и использование QML (QQuickWidget). Также доступны элементы для обработки жестов (QGestureRecognizer), системные инструменты (QSystemTrayIcon, QSizeGrip, QRubberBand), средства помощи (QWhatsThis) и поддержки доступности (QAccessibleWidget). Графическая система Qt включает мощные инструменты для работы с двухмерной и трехмерной графикой через QGraphicsView/QGraphicsScene, OpenGL и низкоуровневые API (Vulkan, Metal, Direct3D). Это делает возможным создание сложных визуальных эффектов и анимаций.

Каждый виджет в Qt имеет богатый набор свойств, методов и сигналов, что позволяет не только изменять внешний вид и поведение, но и глубоко интегрировать элементы в логику приложения. Внешний вид можно гибко настраивать с помощью стилей (QStyle, QSS), аналогичных CSS, а иерархия виджетов обеспечивает эффективное управление памятью и жизненным циклом объектов.

1.7 Архитектура модель–представление

Qt предлагает гибкий подход к организации данных и их отображения через архитектуру «Модель–Представление», которая разделяет логику хранения данных (модель) и способ их отображения (представление). Основные компоненты этой архитектуры включают: модель (QAbstractItemModel), отвечающую за управление данными и их структурой; представление (QAbstractItemView), которое определяет визуализацию данных; и делегат (QStyledItemDelegate), контролирующий отрисовку и редактирование элементов интерфейса. Такое разделение позволяет изменять логику работы с данными независимо от их отображения, что повышает гибкость и масштабируемость приложения.

Преимущества такой архитектуры заключаются в следующем:

- Единое хранилище данных для нескольких представлений: модель может одновременно использоваться разными представлениями (например, таблицей, списком и деревом), что исключает дублирование данных и обеспечивает синхронное обновление.
- Простое подключение к базам данных: встроенные классы (например, QSqlTableModel) позволяют легко интегрировать данные из SQL-баз без необходимости ручного парсинга.
- Гибкость и расширяемость: можно создавать кастомные модели (наследуясь

от `QAbstractItemModel`) и делегаты для нестандартных сценариев отображения или редактирования.

- Наличие стандартных моделей для типовых задач: Qt включает готовые модели, оптимизированные под распространённые структуры данных.
- Наличие готовых классов представлений для отображения данных в различных форматах.

Например, `QStringListModel` – это одномерный список строк, который можно использовать с `QListView`:

```
QStringListModel* model = new QStringListModel({"Пункт 1", "Пункт 2"});
QListView* view = new QListView(this);
view->setModel(model);
```

`QStandardItemModel` — произвольная древовидная структура (узлы могут содержать текст, иконки, пользовательские данные), а `QFileSystemModel` — модуль для доступа к файловой системе с автоматическим обновлением при изменениях:

```
QFileSystemModel *model = new QFileSystemModel;
model->setRootPath(QDir::homePath()); // Корневая директория
QTreeView *view = new QTreeView;
view->setModel(model); // Подключение модели к представлению
view->show();
```

Здесь представление `QTreeView` автоматически отобразит содержимое файловой системы, а модель будет отслеживать изменения (например, новые файлы). Также имеется `QSqlTableModel` для работы с таблицами SQL-баз с поддержкой редактирования и фильтрации.

Для централизованного управления выделением используется `QItemSelectionModel`. Это позволяет синхронизировать выделение между несколькими представлениями одной модели:

```
QItemSelectionModel* selection = new QItemSelectionModel(model);
view1->setSelectionModel(selection);
view2->setSelectionModel(selection);
```

В Qt при работе с представлениями данных важную роль играет концепция делегатов — классов, отвечающих за отрисовку и редактирование элементов интерфейса. Каждое стандартное представление (например, `QTableView`) имеет свой собственный делегат по умолчанию. Это позволяет очень просто создавать представления для любых нужд без написания большого количества нового кода. Существует два основных класса для реализации этой функциональности: `QItemDelegate` и `QStyledItemDelegate`.

`QItemDelegate` — базовый класс делегата, который предоставляет минимальный набор методов для отображения и редактирования элементов. Однако он

не учитывает текущий стиль приложения, что может привести к несоответствию внешнего вида элементов общему дизайну.

`QStyledItemDelegate` — более современный и предпочтительный класс, используемый по умолчанию в Qt, он учитывает текущий стиль оформления (`QStyle`), поддерживает работу с таблицами стилей Qt и обеспечивает корректное отображение элементов в разных платформах и темах. Поэтому в случае реализации пользовательских делегатов рекомендуется использовать `QStyledItemDelegate` в качестве базового класса, также он подойдет при работе с таблицами стилей Qt.

Предположим, мы хотим сделать так, чтобы при наведении курсора мыши на элемент таблицы он подсвечивался цветом. Для этого создадим свой делегат, унаследованный от `QStyledItemDelegate`:

```
class HighlightOnHoverDelegate : public QStyledItemDelegate {
    Q_OBJECT
public:
    using QStyledItemDelegate::QStyledItemDelegate;
    void paint(QPainter *painter, const QStyleOptionViewItem &option,
const QModelIndex &index) const override {
        QStyleOptionViewItem opt = option;
        initStyleOption(&opt, index);
        // Если курсор над элементом — меняем фон
        if (opt.state & QStyle::State_MouseOver) {
            painter->fillRect(opt.rect, QColor(240, 240, 255)); // цвет фона
        }
        //переопределение отрисовки элементов
        QStyledItemDelegate::paint(painter, opt, index);
    }
};
```

Так как данный делегат наследуется от стандартного делегата Qt, то вся базовая функциональность делегата (редактирование, стандартное отображение) сохраняется. Чтобы использовать этот делегат, включаем реакцию на событие наведения мыши (`hover`) и устанавливаем делегат:

```
tableView->setAttribute(Qt::WA_Hover);
tableView->setItemDelegate(new HighlightOnHoverDelegate(tableView));
```

Делегаты в Qt играют важную роль не только в отрисовке и редактировании элементов интерфейса, но и в обеспечении гибкости и функциональности представлений. С их помощью можно тонко настраивать внешний вид элементов — например, изменять цвет фона или текста, задавать определённый шрифт и расположение содержимого внутри ячейки. Это позволяет адаптировать отображение данных под стили приложения или улучшать пользовательский опыт за счёт наглядного представления информации. Делегаты также позволяют ограничивать типы вводимых данных — например, разрешить ввод только чисел или дат, что особенно полезно при работе с формами, требующими строгого

формата ввода. Ещё одной важной возможностью делегатов является поддержка сложных виджетов в ячейках: с помощью них можно добавлять элементы управления, такие как выпадающие списки `QComboBox`, флажки `QCheckBox` и другие интерактивные компоненты прямо внутрь таблиц или списков, расширяя тем самым возможности пользовательского интерфейса. Помимо этого, делегаты обеспечивают валидацию данных — проверку значения перед его сохранением, чтобы убедиться в его корректности и соответствии заданным критериям.

1.8 Создание меню приложения

Меню является важной и неотъемлемой частью практически любого приложения с графическим интерфейсом. Главное меню, как правило, находится в верхней части главного окна приложения и группирует команды, которые пользователь может вызывать в приложении. В библиотеке QT меню реализуется классом `QMenu`. Этот класс определен в заголовочном файле `QMenu`. Основное назначение класса — это размещение команд в меню. Каждая команда представляет объект действия — класс `QAction`. Все действия или сами команды меню могут быть соединены со слотами для исполнения соответствующего кода при выборе команды пользователем.[3] Например, если пользователь выбрал команду меню, то будет послан сигнал `triggered()`, и если вам потребуется в этот момент выполнить какие-либо операции, то данный сигнал действия нужно соединить с соответствующим слотом.

Рассмотрим на примере создание меню в Qt Creator. Предположим, что создан проект с формой `MainWindow`. Открыв режим дизайна, можно сразу увидеть подсказку «Пишите здесь» (рис. 1.8.1).

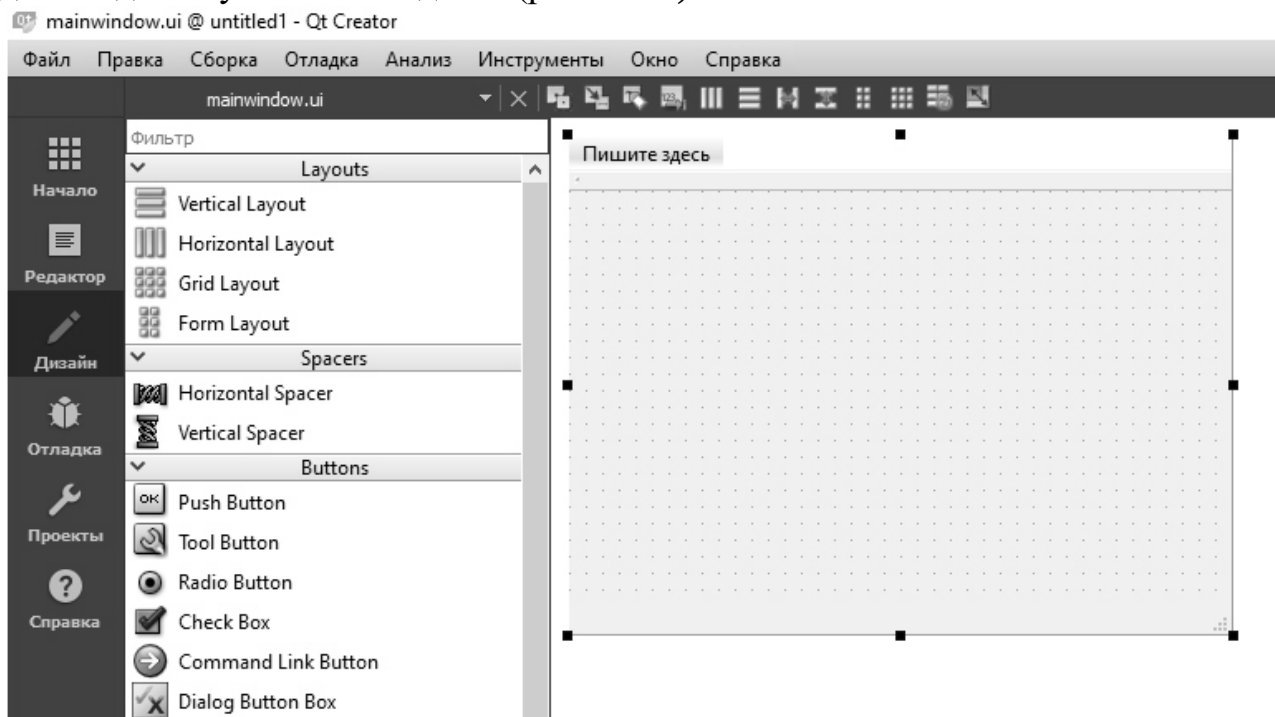


Рисунок 1.8.1 – Подсказка для создания меню

При щелчке мышью на данную подсказку, начнется ввод пунктов меню (рис. 1.8.2).

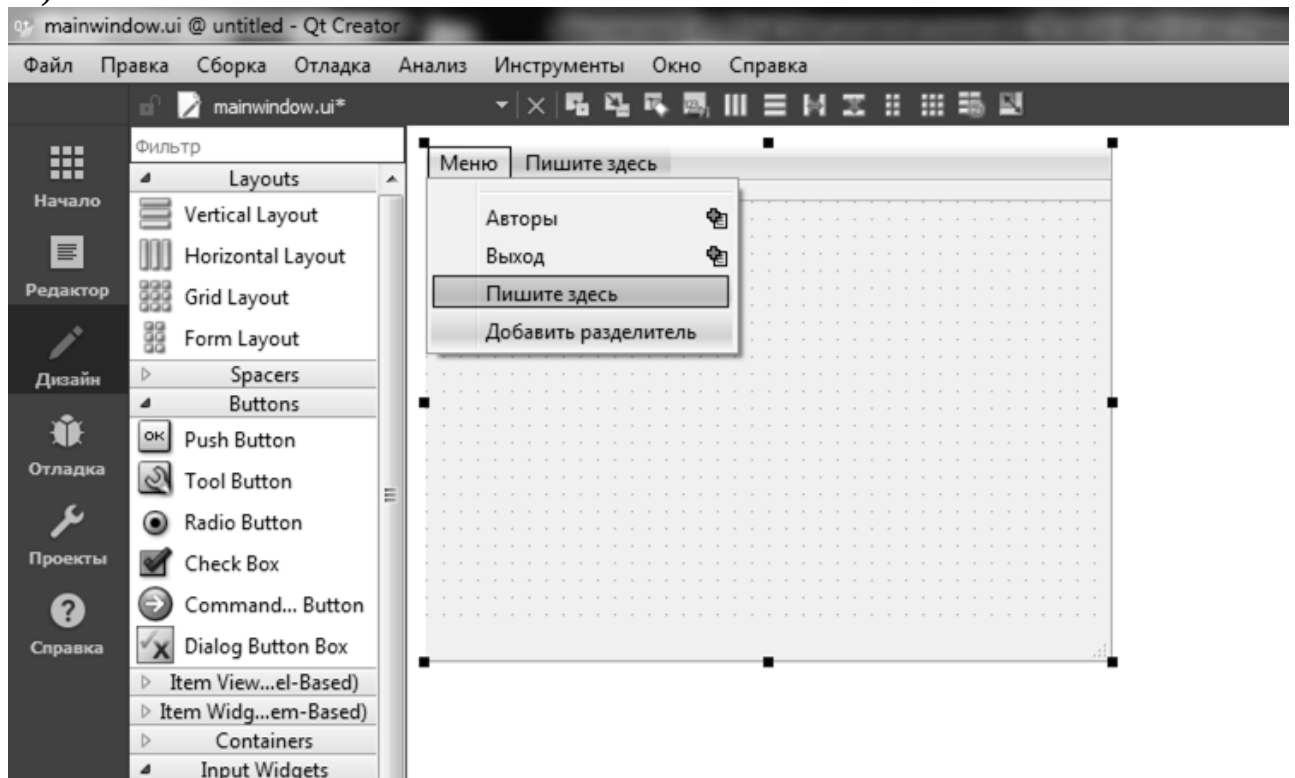


Рисунок 1.8.2 – Ввод пунктов меню

Создадим два пункта: «Авторы» и «Выход». Для создания действия по пункту «Выход» достаточно будет подключить стандартный слот для закрытия окна приложения. Для этого необходимо перейти в режим редактирования сигналов и слотов и, выбрав в нижнем окне «редактор сигналов и слотов», нажать на значок с зеленым плюсом (рис. 1.8.3). Затем, нужно выбрать в колонке «Отправитель» действие action_3, соответствующее пункту меню «Выход», в колонке «Сигнал» выбрать сигнал triggered(), в колонке «Получатель» выбрать MainWindow, а в колонке «Слот» выбрать стандартный слот close().

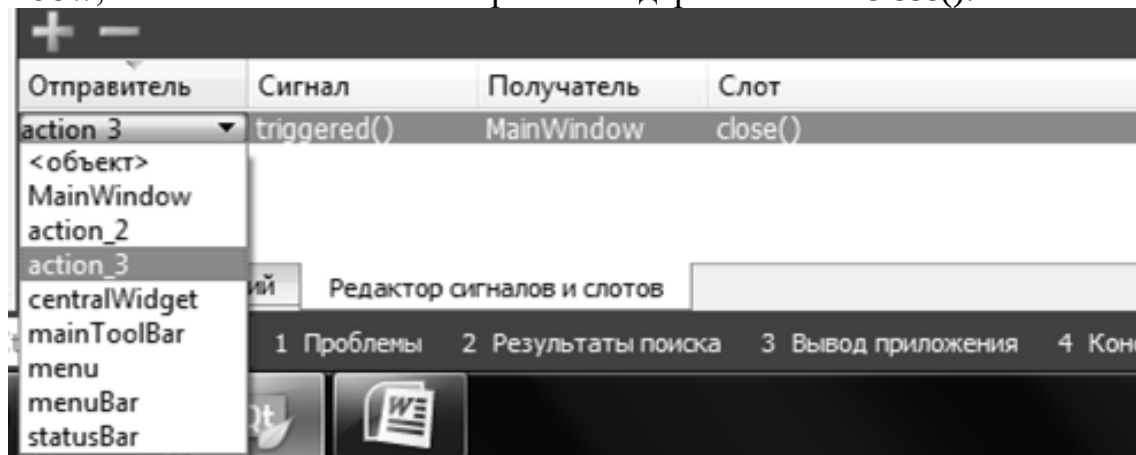


Рисунок 1.8.3 – Редактор сигналов и слотов

Для программирования действия по пункту меню «Авторы» предполагается создание своего собственного слота (функции). В конструкторе MainWindow созданный слот необходимо подключить с помощью функции connect к соответствующему action_2 пункта меню «Авторы» по сигналу triggered().

Меню приложения можно создавать не только с помощью графического конструктора, но и непосредственно программным способом, прямо в коде. Для этого откроем файл mainwindow.cpp и рассмотрим реализацию создания меню внутри конструктора класса MainWindow. Для описания действий, доступных пользователю, в Qt используется класс QAction. Он позволяет объединить в одном объекте несколько элементов, связанных с тем или иным функциональным пунктом интерфейса. К таким элементам относятся:

- отображаемый текст действия;
- всплывающая подсказка (tooltip);
- контекстная помощь («Что это?»);
- сочетание «горячих» клавиш;
- графический значок;
- текст, отображаемый рядом с иконкой;
- сообщение, выводимое в строке состояния;
- заданный шрифт для отображения.

Для каждого из этих параметров в классе QAction предусмотрен отдельный метод: setText(), setToolTip(), setWhatsThis(), setShortcut(), setIcon(), setIconText(), setStatusTip() и setFont() соответственно.

После настройки действия его можно добавить к нужному элементу интерфейса, например, к выпадающему меню. Для этого используется метод addAction(). В данном случае мы будем работать с виджетом QMenu, который представляет собой меню, способное быть частью строки меню или отдельным контекстным меню.

Такой подход даёт возможность гибко управлять содержимым меню и обеспечивает полный контроль над внешним видом и поведением пунктов меню на уровне программного кода. Создадим действие, например, pactOpen для открытия файла и добавим его в меню:

```
43      QAction* pactOpen = new QAction("file open action", 0);
44      pactOpen->setText("&Открыть");
45      pactOpen->setShortcut(QKeySequence("CTRL+A"));
46      pactOpen->setToolTip("Открытие документа");
47      pactOpen->setStatusTip("Открыть файл");
48      pactOpen->setWhatsThis("Открыть файл");
49      pactOpen->setIcon(QPixmap("1.png"));
50      connect(pactOpen, SIGNAL(triggered()), SLOT(slotOpen()));
51      QMenu* pmnuFile=new QMenu("&Файл");
52      pmnuFile->addAction(pactOpen);
53      menuBar()->addMenu(pmnuFile);
```

В строке 43 создаем новое действие `raiseOpen` с текстом «file open action» и указанием на отсутствие родительского виджета (0 во втором параметре), в строке 44 задаем текст действия, а в строке 45 присваиваем комбинацию клавиш для быстрого доступа. Строки 46-48 устанавливают текст всплывающей подсказки, текст подсказки о состоянии и текст подсказки «Что это?», а в строке 49 устанавливается значок `1.png` для данного действия. Строка 50 содержит вызов функции `connect` для связывания созданного действия со слотом по сигналу `triggered()`. Подразумевается, что слот `slotOpen()` реализует операции по открытию файла. В строке 51 создается меню с пунктом «Файл», а строка 52 добавляет действие `raiseOpen` в этот пункт меню. В строке 53 созданное меню добавляется на панель меню главного окна приложения. Таким образом, будет создано главное меню с пунктом «Файл», по нажатию на который будет вызван слот для обработки этого действия. Аналогично действия можно добавлять к другим виджетам с помощью метода `addAction()` и создавать контекстное меню, установив методом `setContextMenuPolicy(Qt::ActionsContextMenu)` стратегию контекстного меню для виджета.

Для создания контекстного меню к виджету (например, `QPushButton`, `QLabel`, `QListWidget` и т. д.) в Qt Designer необходимо выполнить следующие шаги:

- 1) Открыть файл `.ui` вашего окна в Qt Designer.
- 2) Нажать правой кнопкой мыши на виджете и выбрать «Change context menu».
- 3) Появится редактор меню, в котором нужно щелкнуть правой кнопкой мыши внутри поля и выбрать "Insert Item", чтобы добавить пункт меню.
- 4) Добавить нужное количество пунктов, установить им иконки, текст, горячие клавиши и прочее по необходимости.

В результате этих шагов автоматически создадутся действия (`QAction`) и подключатся к событию вызова контекстного меню, а при запуске приложения по нажатию правой кнопки мыши на этот виджет будет появляться созданное меню.

Для большей гибкости рекомендуется создавать контекстное меню программно. Для этого в приложении на Qt необходимо воспользоваться классами `QMenu` и `QAction` и в классе главного окна переопределить метод `contextMenuEvent`, который срабатывает при нажатии правой кнопки мыши. Внутри этого метода создаётся объект меню и добавляются пункты в виде действий `QAction`. Каждое действие может выполнять определённую задачу, например, выводить сообщение или закрывать приложение. Связь между действием и реакцией на него устанавливается через сигнал `triggered`. После формирования меню оно отображается в точке щелчка с помощью метода `exec()`:

```
void MainWindow::contextMenuEvent(QContextMenuEvent *event)
{
    QMenu menu(this);
    menu.addAction("Открыть", this, []{ qDebug() << "Открыто"; });
    menu.addAction("Выход", this, &MainWindow::close);
    menu.exec(event->globalPos());
}
```

Представленный код обрабатывает событие нажатия правой кнопки мыши, создавая при этом простое контекстное меню с двумя пунктами: первый выводит сообщение в консоль, а второй закрывает приложение. Меню отображается в точке щелчка и предназначен для быстрого тестирования и понимания базового механизма работы контекстных меню в Qt.

1.9 Создание диалоговых окон

Диалоговое окно – это центральный элемент, обеспечивающий взаимодействие между пользователем и приложением. Диалоговые окна всегда являются виджетами верхнего уровня и имеют свой заголовок. Их можно разбить на три основные категории:

- стандартные диалоговые окна;
- окна сообщений;
- собственные диалоговые окна.

Диалоговые окна играют важную роль в организации взаимодействия пользователя с приложением: они служат основным средством для получения данных от пользователя, подтверждения действий или отображения важной информации. Как правило, приложения содержат главное окно, содержащее строку меню и панель инструментов, а также одно или несколько диалоговых окон, каждое из которых решает свою конкретную задачу. Однако некоторые программы могут быть реализованы в виде единственного диалогового окна, которое становится центральным элементом взаимодействия с пользователем. В таком случае оно напрямую принимает данные пользователя, на основе которых выполняются необходимые действия, что удобно для небольших утилит или мастеров настройки. Такие приложения называются диалоговыми приложениями [3].

В Qt базовым классом для всех видов диалоговых окон является класс `QDialog`. Он содержит функционал, необходимый для реализации диалогового взаимодействия с пользователем: настройку модальности, управление кнопками, поддержку макетов и многое другое. Все остальные диалоговые окна — как стандартные, так и пользовательские — наследуются от этого класса. При этом диалоговые окна условно делятся на два типа:

- Модальные — такие окна блокируют работу основного интерфейса до тех пор, пока пользователь не завершит взаимодействие с ними. Обычно используются для критически важных операций, таких как сохранение данных или выбор действия.
- Немодальные — работают независимо от главного окна приложения, позволяя пользователю взаимодействовать как с диалогом, так и с основным интерфейсом одновременно. Примером может служить палитра цветов или окно поиска.

Режим работы диалогового окна можно задать программно с помощью метода `setModal()`, передав в него значение `true` (модальное) или `false` (немодальное). Также можно проверить текущий режим с помощью метода `isModal()`.

Одним из удобных аспектов разработки на Qt является наличие стандартных диалоговых окон, которые уже реализованы и готовы к использованию. Их применение позволяет значительно ускорить процесс создания интерфейса, а также обеспечивает единообразие визуального оформления приложений. Следующие классы отвечают за стандартные диалоговые окна в Qt:

- `QFileDialog` – реализует диалоговое окно выбора файлов и отвечает за создание и работу окон выбора файлов для открытия, сохранения и для выбора директории;
- `QPrintDialog` – реализация диалогового окна выбора и настройки принтера, а также вывода на печать;
- `QColorDialog` – реализует диалоговое окно выбора цвета;
- `QFontDialog` – реализует диалоговое окно выбора шрифта;
- `QInputDialog` – реализация диалогового окна ввода для предоставления пользователю возможности ввести строку или число;
- `QProgressDialog` – реализует диалоговое окно прогресса для информирования пользователя о начале продолжительной операции и визуальной оценки времени работы;
- `QWizard` – реализация диалогового окна мастера для сопровождения пользователя при выполнении им операций, которые требуют непосредственно его участия;

Ещё один тип диалоговых окон — это окна сообщений, которые применяются для вывода информации, предупреждений или ошибок. В Qt за них отвечает класс `QMessageBox`. Такие окна всегда модальные и могут содержать заголовок, текст сообщения, значок и набор кнопок для реакции пользователя (рис. 1.9.1).

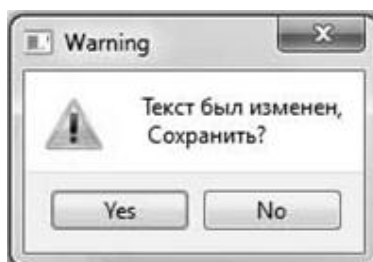


Рисунок 1.9.1 – Окно сообщения

Помимо стандартных решений, разработчики часто создают собственные диалоговые окна, адаптированные под конкретные задачи. Например, для ввода специфических данных или выполнения ограниченной по времени операции. Пользовательские диалоги позволяют гибко настраивать внешний вид, поведение и логику, что делает их универсальным инструментом для взаимодействия с пользователем.

Рассмотрим на примере создание собственного диалогового окна, представляющего собой кнопку, по нажатию на которую отобразится окно для ввода

данных. Для этого понадобится создать свой класс, например StartDialog, который в данном случае будет унаследован от класса кнопки QPushButton. Сначала создадим файлы класса: кликнув правой кнопкой мыши по разделу с заголовочными файлами в режиме редактора QtCreator, нужно выбрать пункт «Добавить новый...» (рис. 1.9.2) и выбрать «Класс C++» (рис.1.9.3).

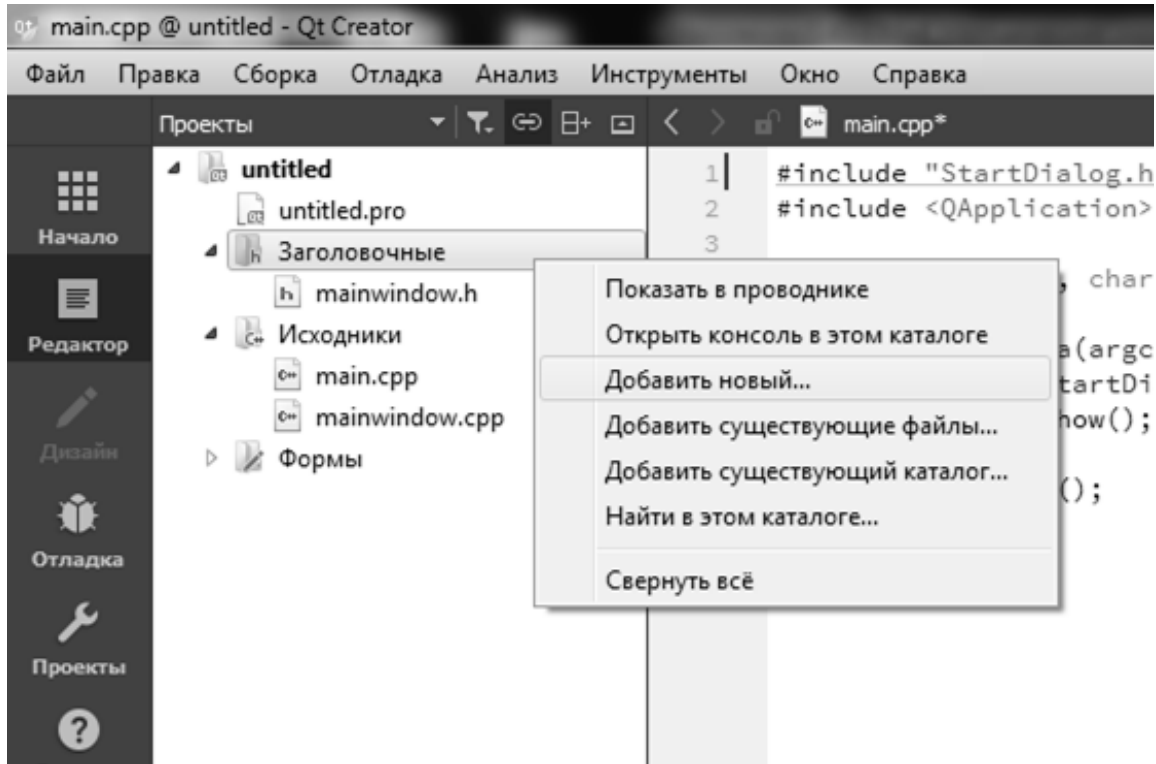


Рисунок 1.9.2 – Добавление файла в проект

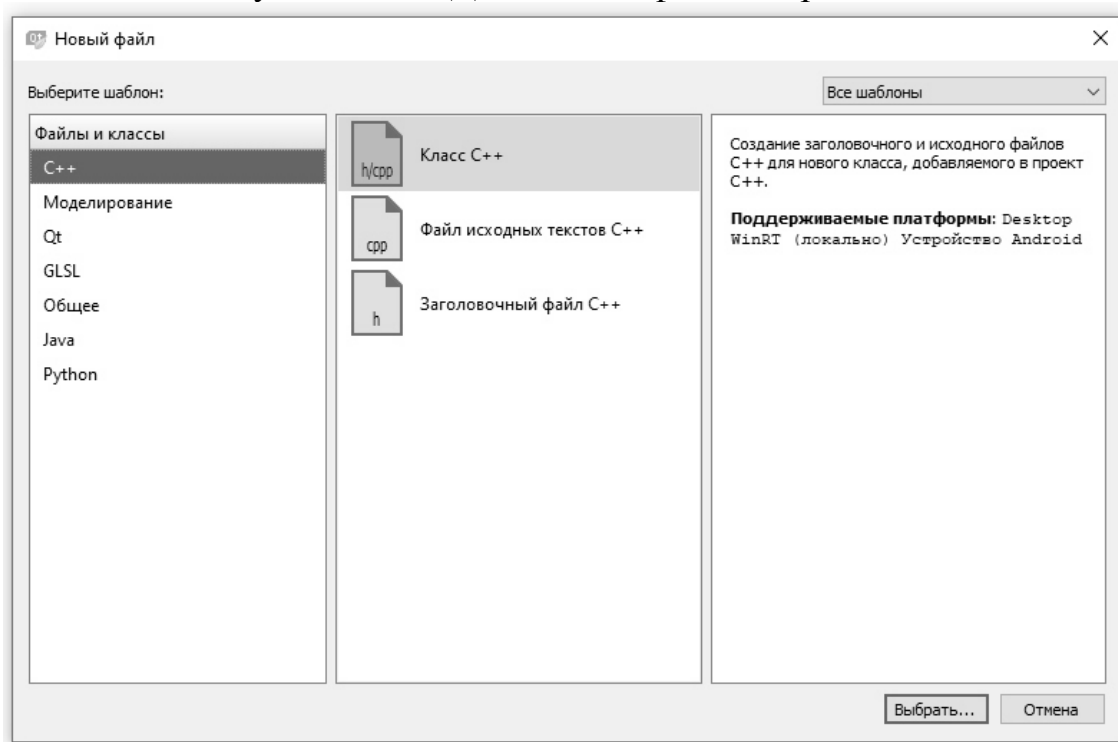


Рисунок 1.9.3 – Создание класса C++

Далее откроется окно для определения класса, в котором необходимо записать имя класса StartDialog (рис. 1.9.4).

Теперь в главной функции файла main.cpp создадим виджет класса StartDialog, предназначенный для запуска диалогового окна:

```
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    StartDialog startDialog;
    startDialog.show();
    return a.exec();
}
```

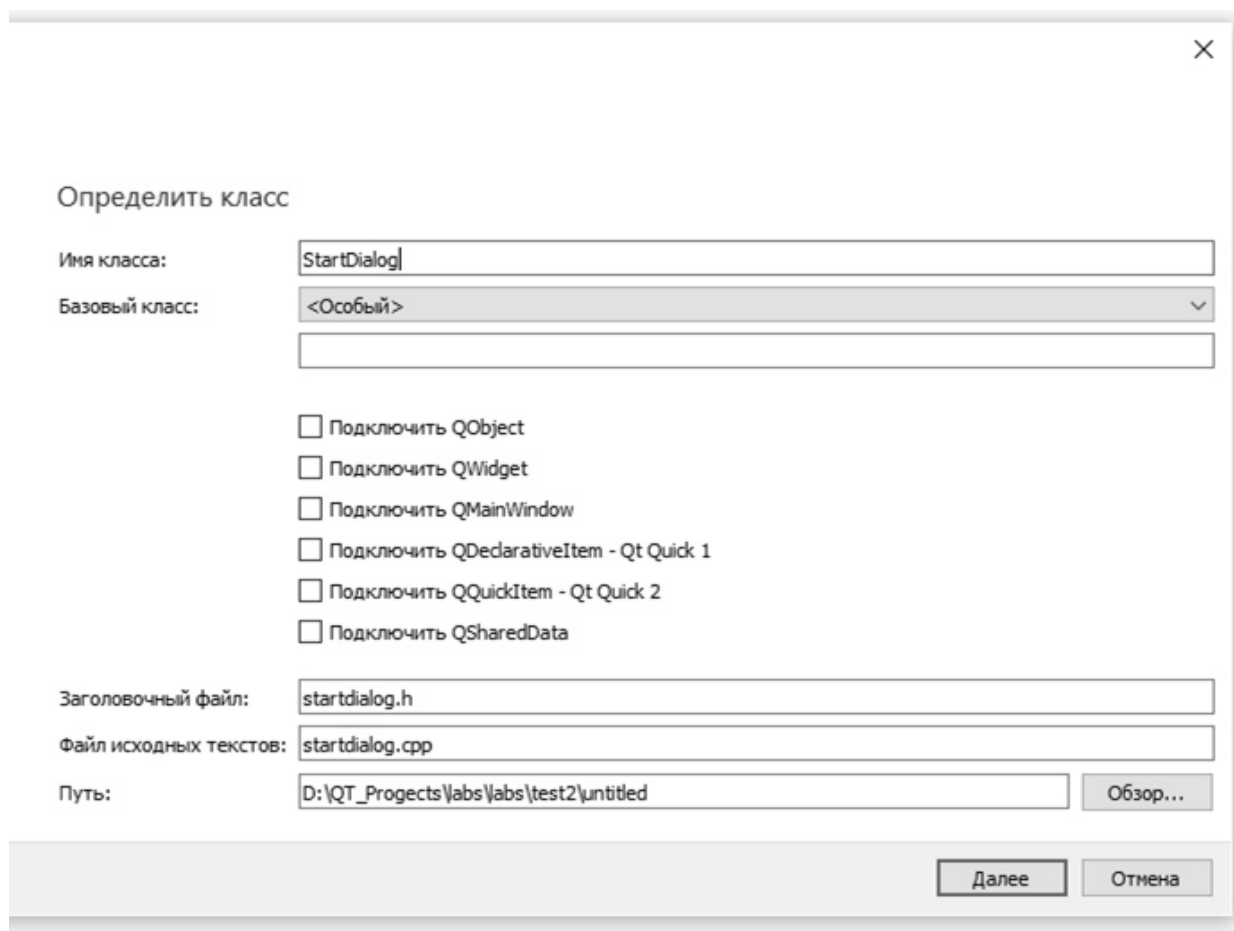


Рисунок 1.9.4 – Добавление названия создаваемого класса

В файле StartDialog.h необходимо указать, что класс StartDialog унаследован от класса кнопки QPushButton. Также здесь сигнал clicked() кнопки соединяется со слотом slotButtonClicked() методом connect():

```
class StartDialog : public QPushButton {
    Q_OBJECT
public:
    StartDialog(QWidget* pwgt = 0) : QPushButton("Нажми", pwgt)
```

```

{
    connect(this, SIGNAL(clicked()), SLOT(slotButtonClicked()));
}

```

Позже мы определим класс диалогового окна InputDialog. А пока в том же файле реализуем слот slotButtonClicked(), в котором создается объект диалогового окна InputDialog:

```

public slots:
    void slotButtonClicked()
    {
        InputDialog* pInputDialog = new InputDialog;
        if (pInputDialog->exec() == QDialog::Accepted) {
            QMessageBox::information(0, "Ваша информация: ", "Имя: " + pInputDialog->firstName() + "\nФамилия: " + pInputDialog->lastName());
        }
        delete pInputDialog; } };

```

Здесь в условии оператора if выполняется запуск диалогового окна, а после его закрытия управление передается основной программе, и метод exec() возвращает значение нажатой пользователем кнопки.

В том случае, если пользователем была нажата кнопка Ок, отобразится информационное окно с введенными в диалоговом окне данными. По завершению метода диалоговое окно нужно удалить, так как у него нет предка. Диалоговые окна, не имеющие предка, отцентрированы на экране.

Далее для создания своего собственного диалогового окна класса InputDialog нужно унаследовать класс QDialog. Класс InputDialog содержит два атрибута: указатели m_ptxtFirstName и m_ptxtLastName на виджеты однострочных текстовых полей и два метода firstName() и lastName(), возвращающие содержимое этих полей:

```

#include <QDialog>
#include <QLineEdit>
class QLineEdit;
class InputDialog : public QDialog
{
    Q_OBJECT
private:
    QLineEdit *m_ptxtFirstName;
    QLineEdit *m_ptxtLastName;
public:
    InputDialog(QWidget* pwgt = 0);
    QString firstName() const;
    QString lastName() const;
};

```

Затем создадим виджеты QLineEdit, Label и PushButton для InputDialog. С помощью функции setBuddy() устанавливается фокус к полям ввода QLineEdit. Модальное диалоговое окно всегда должно содержать кнопку Cancel. Сигналы

clicked() кнопок «Ok» и «Cancel» соединяются со слотами accept() и rejected() соответственно с помощью функции connect(). Это делается для того, чтобы метод exec() возвращал при нажатии кнопки «Ok» значение accepted, а при нажатии на кнопку Cancel – значение rejected:

```
InputDialog::InputDialog(QWidget* pwgt) : QDialog(pwgt)
{
    m_ptxtFirstName = new QLineEdit;
    m_ptxtLastName = new QLineEdit;
    QLabel* plblFirstName = new QLabel("&Имя");
    QLabel* plblLastName = new QLabel("&Фамилия");
    plblFirstName->setBuddy(m_ptxtFirstName);
    plblLastName->setBuddy(m_ptxtLastName);
    QPushButton* pcmdOk = new QPushButton("Ok");
    QPushButton* pcmdCancel = new QPushButton("&Cancel");
    connect(pcmdOk, SIGNAL(clicked()), SLOT(accept()));
    connect(pcmdCancel, SIGNAL(clicked()), SLOT(reject()));
}
```

Теперь необходимо компоновать виджеты на форме. Для этого Qt предоставляет менеджеры компоновки, позволяющие организовать размещение виджетов на поверхности другого виджета. Основой менеджеров компоновки является класс QLayout, от которого унаследованы классы QGridLayout и QVBoxLayout. Класс QGridLayout управляет табличным размещением, а от QVBoxLayout унаследованы два класса: QHBoxLayout и QVBoxLayout – для горизонтального и вертикального размещения соответственно. При помощи метода addWidget() выполняется добавление виджетов в компоновку, а с помощью метода setLayout() компоновка устанавливается в виджете:

```
QGridLayout* ptopLayout = new QGridLayout;
ptopLayout->addWidget(plblFirstName, 0, 0);
ptopLayout->addWidget(plblLastName, 1, 0);
ptopLayout->addWidget(m_ptxtFirstName, 0, 1);
ptopLayout->addWidget(m_ptxtLastName, 1, 1);
ptopLayout->addWidget(pcmdOk, 2, 0);
ptopLayout->addWidget(pcmdCancel, 2, 1);
setLayout(ptopLayout);
```

Метод firstName() возвращает введенное пользователем имя, а метод lastName() – фамилию:

```
QString InputDialog::firstName() const
{
    return m_ptxtFirstName->text();
}
QString InputDialog::lastName() const
{
    return m_ptxtLastName->text();
}
```

При запуске приложения мы увидим кнопку, которая является созданным виджетом нашего класса StartDialog:



Рисунок 1.9.5 – Окно приложения

По нажатию на кнопку откроется собственное диалоговое окно InputDialog:

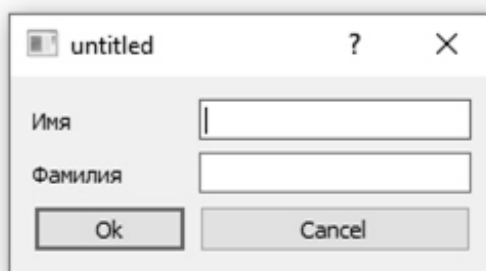


Рисунок 1.9.6 – Окно диалога

Далее после введения данных и нажатия кнопки «Ok» откроется информационное окно с введенными данными:

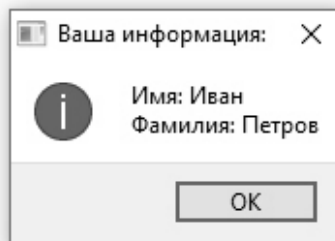


Рисунок 1.9.7 – Информационное окно

Таким образом, было создано диалоговое приложение, использующее собственные диалоговые окна для краткого взаимодействия с пользователем.

При создании собственных диалоговых окон в Qt важно учитывать их взаимодействие с основным окном приложения и другими компонентами интерфейса. Диалоги обычно наследуются от класса QDialog или одного из его потомков, например QMessageBox, и интегрируются в архитектуру приложения через сигналы и слоты, обеспечивая обмен данными между модулями. Взаимодействие диалогового окна с главной формой осуществляется посредством передачи данных при создании диалога — либо через параметры конструктора, либо через публичные методы установки значений. Результат работы диалога (например, нажатие кнопок OK или Cancel) обычно возвращается вызывающе-

му коду с помощью метода `exec()` или `accept()/reject()`, что позволяет организовать логику принятия или отмены внесённых пользователем изменений.

Важно учитывать, что для связи между различными диалогами и основным окном применяется механизм сигналов и слотов: изменения, сделанные в диалоге, могут быть отправлены в главное окно через пользовательские сигналы, которые генерируются при подтверждении действий. Это позволяет централизованно обрабатывать ввод без прямой зависимости между компонентами. Также важно правильно освобождать ресурсы, особенно если диалог содержит динамически создаваемые виджеты или работает с внешними ресурсами, такими как файлы или базы данных.

1.10 Контрольные вопросы

1. Какова структура проекта в Qt и какие файлы входят в его состав?
2. Что представляет собой класс `QAction` и как он используется при создании меню и контекстных команд в Qt?
3. Какие модули библиотеки Qt обеспечивают базовые функции для разработки приложений и чем они отличаются друг от друга?
4. Объясните концепцию механизма сигналов и слотов в Qt. Приведите пример подключения сигнала к слоту.
5. Что такое виджет в системе Qt и какие группы виджетов используются для построения графического интерфейса?
6. Что такое диалоговое окно в Qt и какие виды диалоговых окон предоставляет библиотека? Приведите примеры стандартных диалогов.
7. Что такое архитектура «Модель–Представление» в Qt и какие компоненты в неё входят? Приведите пример использования модели и представления.
8. Как создаются и настраиваются пользовательские диалоговые окна в Qt?
9. Что представляют собой делегаты в Qt и какова их роль в отображении и редактировании элементов данных?
10. Какие инструменты предоставляет Qt для управления расположением элементов интерфейса внутри окна или виджета?

2. МЕТОДИЧЕСКИЕ УКАЗАНИЯ ПО ВЫПОЛНЕНИЮ ПРАКТИЧЕСКИХ РАБОТ В IDE QT CREATOR

2.1 Практическая работа №1

Тема: Модификация приложения калькулятора в среде QtCreator.

Цель работы: познакомиться со средой разработки QtCreator, изучить механизм сигналов и слотов, а также работу с QAction.

Задание:

Клонировать open-source проект <https://github.com/vpapadopou/qt-simple-calculator> и добавить новый функционал:

- Пункт главного меню «Сохранить значение» , который сохраняет значение из дисплея (displayPanel) в список истории.
- Отображение временного сообщения в StatusBar о сохранении значения.
- Пункт меню «История» с подпунктами «Показать», «Скрыть» , который открывает и закрывает боковое окно с сохранёнными значениями и кнопками управления «скрыть» и «очистить» историю.
- Пункт меню «Справка» для вывода информации о работе калькулятора и сведения об авторах.

Порядок выполнения:

- 1) Убедитесь, что проект калькулятора собран и работает.
- 2) Добавьте строку состояния (statusBar) и панель меню (menuBar) на форму (кликнуть правой кнопкой мыши в режиме «Дизайнера» на форме и выбрать «добавить строку состояния», «добавить панель меню»). Создайте пункты меню по заданию (рис. 2.1.1).

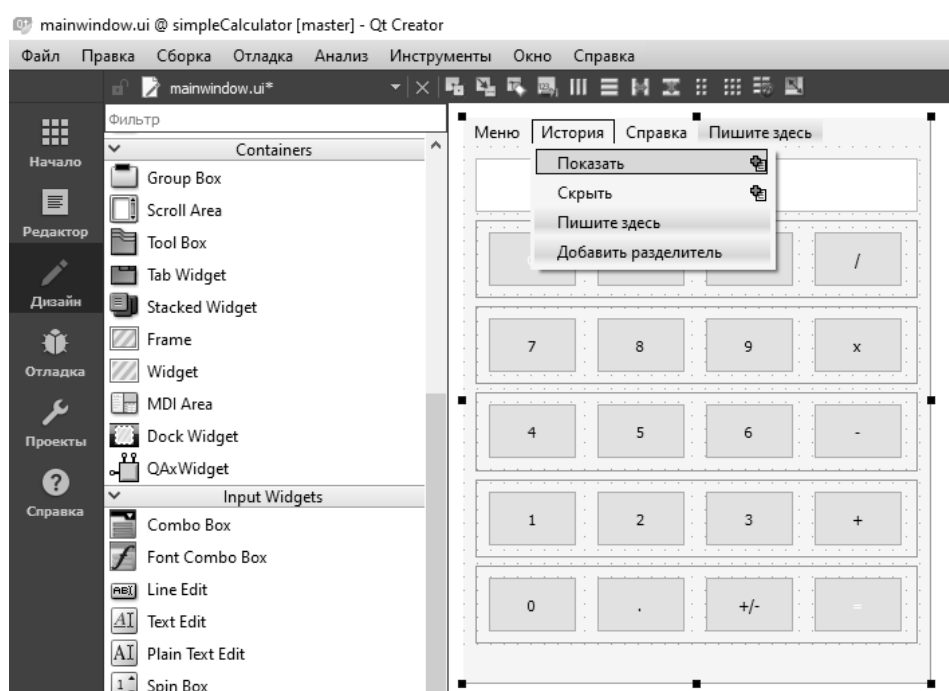


Рисунок 2.1.1 Добавление пунктов меню в режиме дизайнера

Затем задайте новые названия объектам меню в редакторе свойств (рис. 2.1.2).

statusBar	QStatusBar
▼ menuBar	QMenuBar
▼ Menu	QMenu
Save	QAction
▼ History	QMenu
Show_history	QAction
Hide_history	QAction
▼ Info	QMenu
About	QAction

Рисунок 2.1.2 Структура меню в инспекторе объектов

3) Добавьте определение слотов для обработки пунктов меню в файл mainwindow.h (рис. 2.1.3).

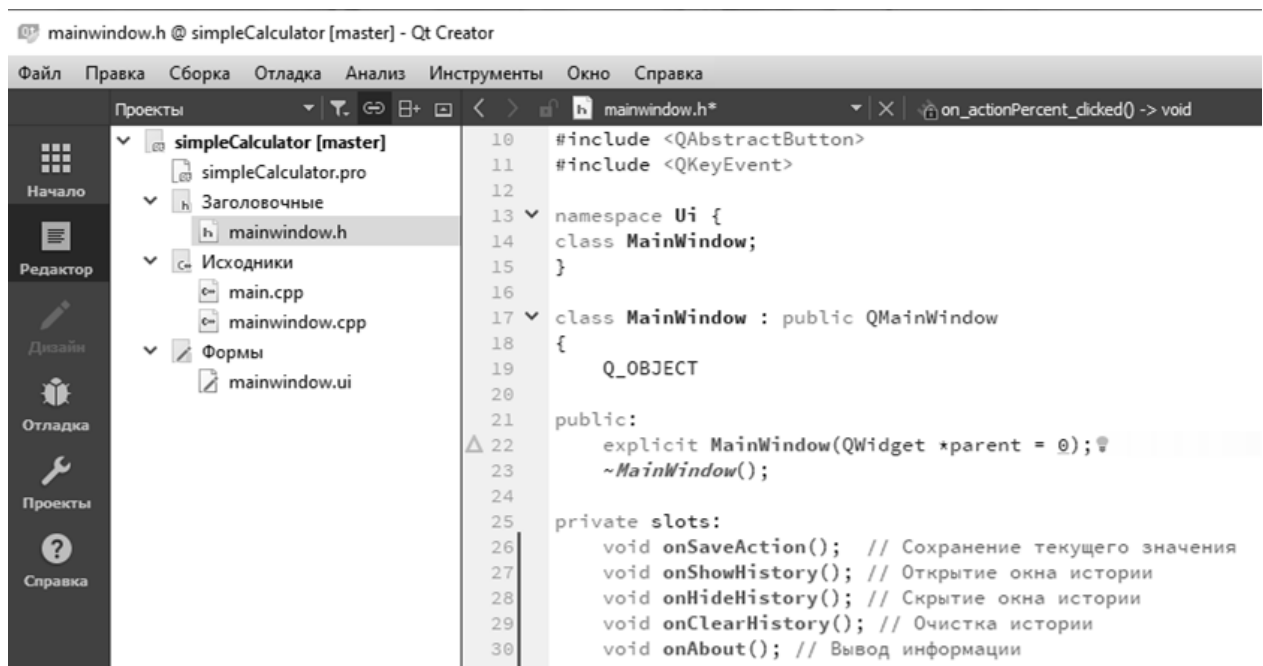


Рисунок 2.1.3. Определение слотов

Также необходимо добавить объявления списка строк (QList для хранения истории) и указателей на элементы интерфейса виджета истории, который далее будет создан. Для временного показа сообщений в statusBar также понадобится объявить указатель на таймер. Объявление переменных для работы с историей вычислений будет таким:

```

QList<QString> historyList; // Хранение истории вычислений
QWidget *historyWidget; // Виджеты для окна истории
QListWidget *historyView;
QPushButton *hideHistoryButton;

```

```
QPushButton *clearHistoryButton;
QVBoxLayout *historyLayout;
QTimer *statusBarTimer; // Таймер для исчезновения сообщения
```

4) В конструкторе MainWindow добавьте создание виджета истории с необходимыми элементами управления. Виджет должен удовлетворять следующим требованиям:

- динамическое создание виджета в коде;
- виджет должен отображаться рядом с основным интерфейсом калькулятора;
- при показе виджета главное окно приложения должно расширяться, а при скрытии – возвращаться к исходной ширине;
- виджет должен содержать список для отображения истории вычислений;
- виджет должен иметь кнопки «скрыть» и «очистить»;
- использование компоновщика;
- нажатие на запись в истории должно обновлять значение на дисплее калькулятора;
- при внесении значения в историю должно отображаться временное сообщение в statusBar;
- пустое значение в список истории не заносится, о чем необходимо уведомлять пользователя в statusBar.

Пример создания виджета истории с использованием вертикального компоновщика:

```
// Создаем виджет истории
historyWidget = new QWidget(this);
historyWidget->setWindowTitle("История");
historyWidget->resize(200, 400);
// Создаем элементы управления
historyView = new QListWidget(historyWidget);
hideHistoryButton = new QPushButton("Скрыть", historyWidget);
clearHistoryButton = new QPushButton("Очистить", historyWidget);
// Вертикальный layout
QVBoxLayout *mainLayout = new QVBoxLayout(historyWidget);
mainLayout->addWidget(hideHistoryButton);
mainLayout->addWidget(clearHistoryButton);
mainLayout->addWidget(historyView);
historyWidget->setLayout(mainLayout);
```

То есть список значений истории (historyList) теперь будет выведен в виджет списка, что допустимо для простых случаев. Для большей гибкости можно воспользоваться паттерном Model/View, о котором речь пойдет позже.

5) В файле mainwindow.cpp добавьте слоты для обработки нажатия кнопок «Скрыть» и «Очистить» для виджета истории:

```
void MainWindow::onHideHistory() {
    if (!historyWidget->isVisible()) return;
```

```

        // Возвращаем окно к исходной ширине
        resize(width() - historyWidget->width(), height());
        // Скрываем виджет истории
        historyWidget->hide();
    }
    void MainWindow::onClearHistory() {
        historyList.clear();
        historyView->clear();
        ui->statusBar->showMessage("История очищена", 3000);
    }

```

6) Добавьте слот для обработки нажатия на значение из списка истории:

```

void MainWindow::onHistoryItemClicked(QListWidgetItem *item) {
    QString value = item->text();
    // Устанавливаем новое значение
    ui->displayPanel->setText(value); }

```

7) На данном этапе у вас создан виджет истории и подготовлены слоты для обработки действий пользователя. Далее необходимо настроить сигнально-слотовые связи для виджета истории. Для этого воспользуйтесь функцией `connect()`: первым параметром укажите элемент интерфейса, который генерирует сигнал, вторым параметром – событие, которое отправляет указанный элемент (сигнал), третьим – объект-получатель сигнала (`this` ссылается на текущий экземпляр класса), а четвертым – слот для обработки события:

```

connect(hideHistoryButton, &QPushButton::clicked, this, &MainWindow::onHideHistory);
connect(clearHistoryButton, &QPushButton::clicked, this, &MainWindow::onClearHistory);
connect(historyView, &QListWidget::itemClicked, this, &MainWindow::onHistoryItemClicked);

```

8) Чтобы виджет истории корректно отображался на форме, можно воспользоваться контейнерным виджетом `QSplitter`. Этот виджет позволяет размещать в себе несколько виджетов рядом друг с другом и не только меняет размеры динамически в зависимости от содержимого, но и позволяет регулировать и изменять размер компонентов вручную. Использование компоновщика `QHBoxLayout` также допускается, но в данном случае будет менее удобным из-за автоматического выравнивания.

Чтобы иметь возможность менять размеры главного окна, необходимо в режиме дизайнера кликнуть на форму и, убедившись, что в инспекторе объектов выбрано `MainWindow`, поменять значение свойства `sizePolicy` на `Preferred` или `Expanding`. Затем в конструкторе `MainWindow` добавить код для создания сплиттера и разместить в нем центральный виджет (`centralWidget`), содержащий первоначальный калькулятор и созданный виджет истории. А с помощью функции `setStretchFactor()` можно указать, как дочерние виджеты внутри сплиттера будут растягиваться при изменении окна: нам нужно, чтобы виджет с калькулятором не менял своего размера. После этого в качестве центрального виджета установите созданный сплиттер:

```

QSplitter *mainSplitter = new QSplitter();
mainSplitter->addWidget(ui->centralWidget);
ui->centralWidget->setMinimumWidth(300);
mainSplitter->addWidget(historyWidget);
mainSplitter->setStretchFactor(0, 1);
setCentralWidget(mainSplitter);

```

9) Последним действием конструктора скройте виджет истории с помощью функции `hide()`.

10) Вернемся к главному меню программы. Вам необходимо реализовать оставшиеся действия для обработки нажатия на пункты меню: сохранение текущего значения истории, открытие окна истории и вывод информации о программе.

Для корректного сохранения значения истории, необходимо взять его из виджета `displayPanel (QLabel)` и внести в список истории, если значение не пустое. Также следует выводить временные сообщения в строке статуса. Например, реализация этих действий может выглядеть следующим образом:

```

void MainWindow::onSaveAction() {
    QString currentValue = ui->displayPanel->text();
    if (currentValue.isEmpty()) {
        ui->statusBar->showMessage("Значение пустое не добавлено в историю", 3000);
        return;
    }
    historyList.append(currentValue);
    ui->statusBar->showMessage("Значение " + currentValue + " добавлено в историю", 3000);
    // Таймер для автоматического скрывтия сообщения
    statusBarTimer = new QTimer(this);
    statusBarTimer->setSingleShot(true);
    connect(statusBarTimer, &QTimer::timeout, this,
    &MainWindow::clearStatusBarMessage);
    statusBarTimer->start(3000);
    updateHistoryList();
}
void MainWindow::clearStatusBarMessage() {
    ui->statusBar->clearMessage();
}
void MainWindow::updateHistoryList() {
    historyView->clear();
    for (const QString& item : qAsConst(historyList)) {
        historyView->addItem(item);
    }
}

```

Теперь калькулятор в развернутом виде должен выглядеть так:

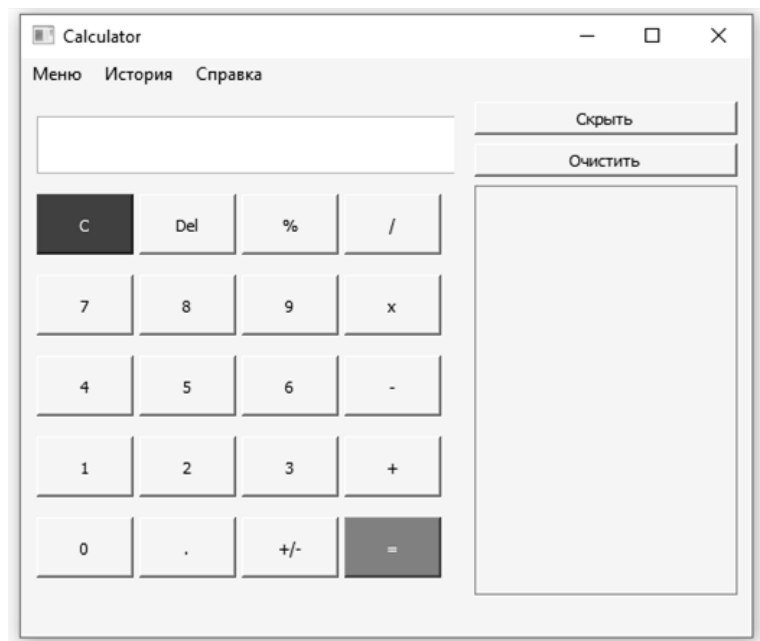


Рисунок 2.1.4 Калькулятор с историей

11) Перейдем к обработке нажатия пункта меню «О программе». Недостаточно просто вывести сообщение о программе – необходимо добавить новую форму для показа справки в удобном для пользователя виде. Окно справки должно содержать в себе следующую информацию:

- инструкцию по использованию калькулятора;
- инструкцию по работе с историей;
- описание горячих клавиш;
- указание лицензии и авторства.

Итак, создайте новый класс формы в Qt Designer. Для этого в списке файлов проекта находим «Формы», правой кнопкой мыши выбираем «Добавить новый», затем выбираем «Класс формы Qt Designer».

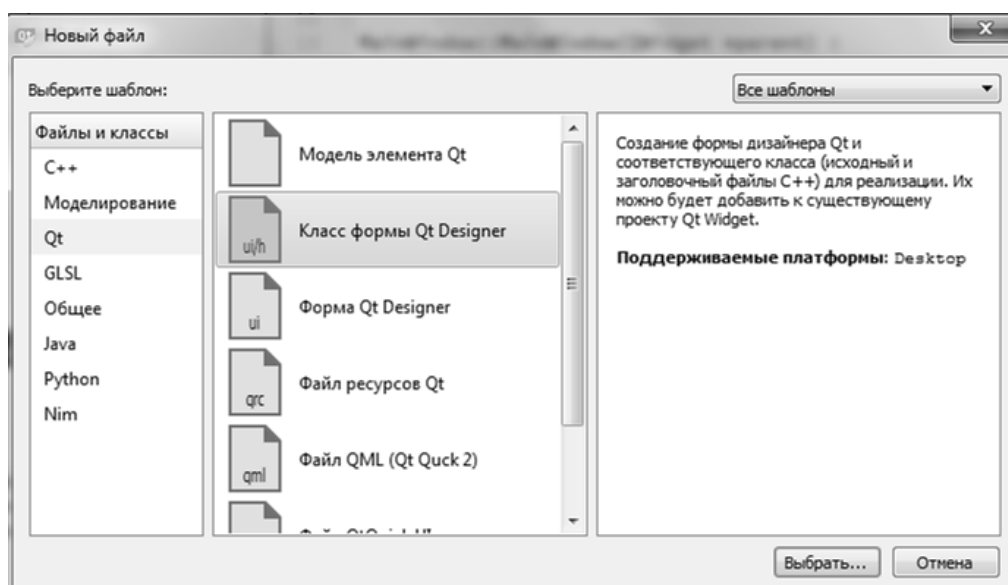


Рисунок 2.1.5 Создание класса формы

Например, пусть класс называется Form, реализуем обработчик нажатия пункта меню «О программе»:

```
void MainWindow::onAbout() {  
    Form *info = new Form();  
    info->show();  
}
```

Таким образом, в обработчике было создано и показано окно информации. Наполнение окна реализуйте самостоятельно с учетом требований.

12) Теперь, когда все обработчики готовы, соедините их с нужными сигналами с помощью функции connect() в конструкторе главного окна:

```
connect(ui->Save, &QAction::triggered, this, &MainWindow::onSaveAction);  
connect(ui->Show_history, &QAction::triggered, this, &MainWindow::onShowHistory);  
connect(ui->Hide_history, &QAction::triggered, this, &MainWindow::onHideHistory);  
connect(ui->About, &QAction::triggered, this, &MainWindow::onAbout);
```

13) Добавьте недостающие горячие клавиши для управления меню и историей.

14) Протестируйте работу полученного приложения, проверив следующие случаи:

- При выборе в меню «История → Показать» размер главного окна расширяется и появляется виджет истории.
- При выборе «Меню → Сохранить значение» и пустом поле displayPanel в строке состояния появляется сообщение о том, что пустое значение не добавлено в историю.
- Сохранение нескольких чисел → история должна расти.
- Нажатие на элемент истории → значение появляется в displayPanel.
- Кнопка "Очистить" → список истории становится пустым.
- Кнопка "Скрыть" → окно истории исчезает, размер главного окна сужается до размера основного калькулятора.
- При повторном показе истории список значений актуальный.

Дополнительное задание: реализовать сохранение истории в файл, загрузку истории из файла.

Контрольные вопросы:

1. Как работает механизм сигналов и слотов?
2. Какие компоновщики (layout) используются в Qt и зачем они нужны?
3. Что такое QSplitter и в чём его отличие от других layout-компонентов?
4. Для чего используется statusBar в приложении? Приведите примеры использования.
5. Как работают горячие клавиши в Qt? Как их назначить для действий?
6. Как реализовать реакцию на клик по элементу списка (QListWidgetItem)?

7. Как сделать так, чтобы главное окно расширялось и сужалось при показе/скрытии боковой панели истории?
8. Как добавить новую форму в проект?
9. Поясните как связана работа с QAction и QMenu в данном проекте.
10. Когда используются сигналы triggered() и clicked() в Qt?

2.2 Практическая работа №2

Тема: Дополнение демонстрационного приложения для работы с библиотекой подсветки QSourceHighlighter в среде QtCreator.

Цель работы: научиться использовать стандартные диалоговые окна библиотеки Qt и создавать пользовательские диалоги на основе QDialog, а также освоить работу с форматом JSON.

Задание:

Клонировать open-source проект <https://github.com/Waqar144/QSourceHighlite> и добавить новый функционал:

- Пункт главного меню «Файл», содержащий следующие действия:
- "Сохранить как..." → TXT или JSON
- "Открыть как..." → текст или файл JSON
- "Выход" с предложением сохранить изменения перед выходом.
- Пункт главного меню «Правка», содержащий следующие действия:
- "Копировать"
- "Вставить"
- "Вырезать"
- "Очистить"
- "Отменить"
- "Повторить"
- "Поиск" → открытие окна поиска.
- Диалоговое окно «Поиск», в котором содержится поле ввода строки поиска и кнопки «Найти далее», «Заменить», «Заменить все».
- Сохранение документа в формате JSON должно позволять сохранять установленный пользователем язык программирования и текст.
- Загрузка файла формата JSON должна позволять восстанавливать текст и язык программирования в интерфейсе приложения.
- Строка состояния, отображающая временные сообщения о выполненных действиях программы.

Порядок выполнения:

- 1) Соберите проект в Qt Creator и убедитесь, что он запускается. Функционал смены темы оформления данная работа не затрагивает.
- 2) Исправьте метод смены языка в выпадающем списке, оставив в нем только два действия:

```
void MainWindow::languageChanged(const QString &lang) {
    highlighter->setCurrentLanguage(_langStringToEnum.value(lang)); //выбор языка
    highlighter->rehighlight(); //смена подсветки
}
```

3) Добавьте пункты в главное меню по заданию.

4) Ряд действий из меню «Правка» содержится в контекстном меню виджета `plainTextEdit`. Свяжите имеющиеся действия с пунктами главного меню, используя режим редактирования сигналов и слотов:



Рисунок 2.2.1. Редактирование сигналов и слотов

5) Реализуйте сохранение обычного текстового файла, выводя в панель состояния временные сообщения о результатах действия. Например, при успешном сохранении файла нужно вывести информацию о том, какой файл был сохранен.

6) Реализуйте сохранение файла в формате JSON, воспользовавшись классами `QJsonObject` и `QJsonDocument`. Сохранять следует текущий выбранный язык из выпадающего списка и текст из поля ввода:

```
void MainWindow::on_actionJSON_triggered()
{
    QString fileName = QFileDialog::getSaveFileName(this, "Сохранить JSON
файл", "", "JSON файлы (*.json)");
    if (fileName.isEmpty()) return;
    QFile file(fileName);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        ui->statusbar->showMessage("Ошибка: не удалось открыть файл для записи", 3000);
        return;
    }
    QJsonObject root;
    root["language"] = ui->langComboBox->currentText();
}
```



```

    root["text"] = ui->plainTextEdit->toPlainText();
    QJsonDocument doc(root);
    file.write(doc.toJson());
    file.close();
    ui->statusbar->showMessage("Сохранен файл " + fileName, 3000);
}

```

В панель состояния также необходимо выводить временные сообщения о результатах данного действия.

7) Реализуйте открытие текстового файла, выводя временные сообщения в строку состояния о том, какой файл был открыт.

8) Реализуйте открытие файла формата JSON, используя классы `QJsonDocument` и `QJsonParseError`:

```

QJsonParseError error;
QJsonDocument doc = QJsonDocument::fromJson(file.readAll(), &error);
file.close();
if (error.error != QJsonParseError::NoError) {
    ui->statusbar->showMessage("Ошибка парсинга JSON", 3000);
    return;
}

```

Для извлечения данных используйте классы `QJsonObject` и `QJsonValue`, получая корневой JSON объект и извлекая значение по ключу для установки текста и языка в редактор:

```

QJsonObject obj = doc.object();
QString lang = obj["language"].toString();
QString text = obj["text"].toString();
// Установка текста
ui->plainTextEdit->setPlainText(text);
// Установка языка
if (!lang.isEmpty()) {
    ui->langComboBox->setCurrentText(lang);
}

```

9) Реализуйте функционал меню для выхода из приложения с проверкой несохраненных изменений, предлагая пользователю сохранить файл при необходимости. Используйте класс `QMessageBox` для вывода диалогового окна и `QTextDocument::isModified()` для проверки на изменения.

10) Создайте диалог поиска и замены, следуя рекомендациям:

- Добавьте новый класс C++ , например, `SearchDialog`, который будет наследоваться от `QDialog`.

- В заголовочном файле определите необходимые элементы управления, например, `QLineEdit` для ввода строк поиска и замены, `QPushButton` для кнопок "Найти далее", "Заменить" и «Заменить все».

- Данный диалог должен отвечать только за сбор данных для поиска и замены, а действия над текстом реализованы в `MainWindow`. Соответственно в

заголовочный файл диалога добавьте сигналы для передачи данных в главное окно приложения редактора:

```
signals:
    void findNext(const QString &text); //поиск
    void replaceText(const QString &find, const QString &replace); //замена
    void replaceAll(const QString &find, const QString &replace); //заменить все
```

- Реализуйте слоты, которые будут вызываться при нажатии кнопок в диалоговом окне поиска: для кнопки «Найти» обработчик нажатия будет отправлять сигнал findNext главному окну со строкой для поиска в редакторе; для кнопок «Заменить» и «Заменить все» - отправка сигнала replaceText и replaceAll со строками для поиска и замены:

```
void SearchDialog::on_findButton_clicked() {
    emit findNext(findLineEdit->text());
}
void SearchDialog::on_replaceButton_clicked() {
    emit replaceText(findLineEdit->text(), replaceLineEdit->text());
}
void SearchDialog::on_replaceAllButton_clicked() {
    emit replaceAll(findLineEdit->text(), replaceLineEdit->text());
}
```

Также объявите данные слоты в заголовочном файле диалога в секции private slots и свяжите их с соответствующими кнопками функцией connect() в конструкторе диалога.

- Добавьте соответствующие слоты в MainWindow, объявив их в заголовочном файле. Слот для поиска должен искать следующее совпадение с текущей позиции. Если совпадение не найдено – продолжить поиск с начала документа. Если не найдено совпадений во всем документе – выводить об этом сообщение. Пример реализации слота поиска:

```
void MainWindow::onFindText(const QString &text) {
    // Получаем курсор из QPlainTextEdit (текущую позицию ввода)
    QTextCursor cursor = ui->plainTextEdit->textCursor();
    // Получаем весь текст из редактора как QString для поиска
    QString content = ui->plainTextEdit->toPlainText();
    // Ищем текст, начиная с текущей позиции курсора
    int pos = content.indexOf(text, cursor.position(), Qt::CaseSensitive);
    if (pos >= 0) { // Нашли совпадение → устанавливаем курсор на него
        cursor.setPosition(pos);
        cursor.movePosition(QTextCursor::Right, QTextCursor::KeepAnchor,
text.length());
        ui->plainTextEdit->setTextCursor(cursor); //выделяем найденный фрагмент
        ui->statusbar->showMessage(tr("Найдено: ") + text, 3000);
    } else {
        // Совпадений нет от текущей позиции до конца, начинаем поиск сначала
    }
}
```

```

pos = content.indexOf(text, 0, Qt::CaseSensitive);
if (pos >= 0) {
    // Нашли совпадение при повторном поиске с начала
    cursor.setPosition(pos);
    cursor.movePosition(QTextCursor::Right, QTextCursor::KeepAnchor,
text.length());
    ui->plainTextEdit->setTextCursor(cursor);
    ui->statusbar->showMessage("Начали поиск с начала. Найдено: " +
text, 3000);
} else {
    // Ничего не найдено даже после повторного поиска
    QMessageBox::information(this, "Поиск", "Совпадений не найдено");
    ui->statusbar->showMessage(tr("Совпадений не найдено"), 3000);
}
}
}

```

Слот для однократной замены должен заменять одно вхождение текста, заданного пользователем. Сначала нужно проверить, выделено ли нужное слово и выполнить замену, если совпадение уже найдено. Если слово не выделено, то должен производиться поиск следующего вхождения искомой строки с текущей позиции курсора. После замены положение курсора должно обновляться, чтобы пользователь видел, где произошли изменения. Затем нужно обновить подсветку. Пример реализации слота замены:

```

void MainWindow::onReplaceText(const QString &find, const QString &replace) {
    // Получаем указатель на документ
    QTextDocument *doc = ui->plainTextEdit->document();
    // Получаем текущий курсор
    QTextCursor cursor = ui->plainTextEdit->textCursor();
    // Проверяем, есть ли выделенный текст
    QString selected = cursor.selectedText();
}
// Если есть выделение, и оно совпадает с find - заменяем его
if (!selected.isEmpty() && selected == find) {
    cursor.insertText(replace); // Заменяем выделенный текст
    ui->plainTextEdit->setTextCursor(cursor); // Перемещаем курсор
    return;
}
int pos = cursor.position();
// Ищем следующее вхождение с учетом регистра
QTextCursor found = doc->find(find, pos, QTextDocument::FindCaseSensitively);
if (!found.isNull()) {
    // Совпадение найдено - сохраняем начальную и конечную позиции курсора
    int start = found.selectionStart();
    int end = found.selectionEnd();
    // Замена найденного текста
    found.insertText(replace);
    // Создаем новый курсор, чтобы переместить выделение на место замены
    QTextCursor newCursor = ui->plainTextEdit->textCursor();
    // Устанавливаем курсор на начало замены
    newCursor.setPosition(start);
}

```

```

        // Выделяем новую строку длиной replace.length()
        newCursor.movePosition(QTextCursor::Right, QTextCursor::KeepAnchor, re-
place.length());
        // Перемещаем курсор в UI
        ui->plainTextEdit->setTextCursor(newCursor);
    } else {
        QMessageBox::information(this, "Поиск", "Совпадений не найдено");
    }
    highlighter->rehighlight(); // обновляем подсветку

```

Слот для замены всех вхождений выполняет замену заданной строки и обновляет содержимое редактора, а также выводит сообщение в строку состояния:

```

void MainWindow::onReplaceAll(const QString &find, const QString &replace) {
    QString content = ui->plainTextEdit->toPlainText();
    // Заменяем все вхождения find на replace с учётом регистра
    QString newContent = content.replace(find, replace, Qt::CaseSensitive);
    // Устанавливаем обновлённый текст обратно в редактор
    ui->plainTextEdit->setPlainText(newContent);
    highlighter->rehighlight();
    QString message = (QString("Все вхождения '%1' заменены на '%2'").arg(find).arg(replace));
    ui->statusbar->showMessage(message, 3000);
}

```

- Реализуйте обработчик вызова диалога поиска из главного меню программы. В данном обработчике необходимо создать экземпляр реализованного диалогового окна и добавить связи сигналов диалога со слотами главного окна:

```

void MainWindow::on_action_10_triggered()
{
    SearchDialog dialog(this);
    connect(&dialog, &SearchDialog::findNext, this, &MainWindow::onFindText);
    connect(&dialog, &SearchDialog::replaceText, this, &MainWindow::onReplaceText);
    connect(&dialog, &SearchDialog::replaceAll, this, &MainWindow::onReplaceAll);
    dialog.exec(); // показываем модальное окно
}

```

11) Протестируйте работу полученного приложения, проверив следующие случаи:

- Сохраните файл в формате .txt и .json и убедитесь, что statusBar выдает сообщение о пути сохранения.
- Проверьте, что в .json файле сохраняются выбранный язык программирования и сам текст., в в .txt файле сохраняется только текст без метаданных.
- Откройте .txt и .json файлы в редакторе и убедитесь, что при открытии .json язык программирования и соответствующая подсветка восстанавливается, а текст корректно загружается в редактор.
- Если вы открываете поврежденный файла JSON, то должно появиться сообщение об ошибке.

- Поменяйте язык в выпадающем списке и убедитесь, что ключевые слова, строки, комментарии и числовые литералы подсвечиваются правильно.
- Откройте любой текст в редакторе и нажмите "Поиск" в меню "Правка", введите строку для поиска и проверьте работает ли кнопка "Найти далее", подсвечивается ли найденный текст, возможна ли замена одного фрагмента и всех вхождений
- Проверьте, что при достижении конца документа поиск продолжается с начала.
- Внесите изменения в текст, но не сохраняйте его, нажмите "Выход" . Должно появиться окно с вопросом о сохранении документа. При выборе "Да" → открывается диалог сохранения, "Нет" → приложение закрывается без сохранения, "Отмена" → выход отменяется, возврат к редактору.
- Проверьте, что все остальные пункты меню работают как ожидается: кнопки "Копировать", "Вставить", "Вырезать", "Очистить" работают как в контекстном меню.
- Проверьте работу statusbar: временные сообщения должны появляться и исчезать через 3 секунды.

Дополнительное задание: Добавить иконки к пунктам главного меню, а также реализовать быструю панель инструментов (toolbar), содержащую часто используемые действия: открытие файла, сохранение, поиск.

Контрольные вопросы:

1. Как работает механизм подсветки синтаксиса в QSyntaxHighlighter?
2. Что делает метод rehighlight() и почему он важен при изменении языка или текста?
3. Какие сигналы и слоты используются в диалоге поиска и замены?
4. Что хранится в JSON-файле при сохранении документа?
5. Как восстанавливается язык программирования при открытии JSON-файла?
6. Как работает связь между ComboBox и подсветкой кода?
7. Какие классы Qt используются для создания пользовательского диалога поиска и замены?
8. Как применяется QTextCursor в данном проекте?
9. Как проверить, были ли сделаны изменения в документе перед выходом?
10. Какие классы и методы Qt используются для работы с JSON?

2.3 Практическая работа №3

Тема: Расширение списка тем оформления в приложении для работы с библиотекой подсветки QSourceHighlighter в среде QtCreator.

Цель работы: Получить практические навыки расширения функциональности приложения на основе библиотеки Qt, научиться работать с мета-объектной системой Qt.

Задание:

- Добавить в проект редактора с подсветкой кода QSourceHighlighter две новые темы оформления.
- Предусмотреть динамическую смену оформления, включая фон всего редактора в зависимости от выбранной темы.
- При сохранении документа в формате JSON сохранять также тему с фоном и восстанавливать ее при открытии файла.

Порядок выполнения:

1) Откройте файл `qsourcehighlighterthemes.cpp`, реализацию функции `theme()` и обратите внимание, что там задана всего одна тема `Monokai`.

2) Откройте файл `qsourcehighlighter.h` и добавьте в перечисление `Themes` новые темы, например темную и светлую:

```
enum Themes {  
    Monokai = 1,  
    DarkTheme = 2,  
    LightTheme = 3,  
};
```

3) Реализуйте эти темы в `qsourcehighlighterthemes.cpp` аналогично функции `monokai()`. Данная функция возвращает ассоциативный массив (словарь) `QHash<Token, QTextCharFormat>`, который связывает токен (тип текста для форматирования) с его форматом. То есть для реализации темы необходимо определить цвета и стили для каждого типа токенов.

4) Добавьте вызов новых тем в метод `theme()`.

5) Откройте файл `mainwindow.cpp` и добавьте новые темы для выбора в интерфейс, дописав метод `initThemesComboBox()`. Начальным значением по умолчанию в выпадающий список тем задайте значение «default», при выборе которого должен быть использован стандартный стиль текста Qt.

6) Для смены фона виджета редактора добавьте метод `applyEditorBackground()`, который будет вызываться из `themeChanged()` с выбранной из списка темой. Объявите его в заголовочном файле `mainwindow.h` следующим образом:

```
void applyEditorBackground(QSourceHighlighter::Themes theme);
```

Реализация метода смены фона редактора соответственно теме в файле `mainwindow.cpp`:

```
void MainWindow::applyEditorBackground(QSourceHighlighter::Themes theme) {  
    // Получаем форматы для текущей темы  
    auto themeFormats = QSourceHighlighterTheme::theme(theme);  
    // Берём формат блока – он содержит фон всей строки  
    QTextCharFormat blockFormat = themeFormats.value(QSourceHighlighter::CodeBlock);
```

```

// Получаем цвет фона. Если его нет – используем белый
QColor bgColor = blockFormat.background().color();
if (!bgColor.isValid()) {
    bgColor = Qt::white;
}
// Устанавливаем стиль только для фона
QString styleSheet = QString("QPlainTextEdit { background-color: %1; }").
arg(bgColor.name());
// Применяем стиль ко всему редактору
ui->plainTextEdit->setStyleSheet(styleSheet);
}

```

7) Доработайте сохранение файла в формате JSON, добавив тему оформления в файл. При открытии соответственно тема и фон должны быть восстановлены вместе с текстом и языком.

8) Протестируйте работу полученного приложения, проверив следующие случаи:

- При запуске приложения отображается стандартная цветовая схема оформления (тема «Default»).
- При открытии текстового файла тема остается текущей.
- При смене темы происходит обновление оформления текста и фона.
- Сохранение/открытие JSON → тема и язык должны сохраниться.

Дополнительное задание: реализовать диалог с возможностью создания пользовательской темы оформления. Тема должна сохраняться в отдельный файл и подгружаться в список тем при каждом запуске редактора.

Контрольные вопросы:

1. Как передается значение из QComboBox в слот выбора темы?
2. Какую роль играет QHash<QSourceHighlighter::Token, QTextCharFormat> в реализации тем оформления?
3. Какие элементы кода окрашиваются в функции описания темы оформления? Перечислите все токены и их назначение.
4. Какой класс в Qt используется для задания форматирования текста, и как он применяется при реализации подсветки синтаксиса через QSourceHighlighter?
5. Как сохранить текущую тему оформления в JSON-файле вместе с текстом и языком программирования?

2.4 Практическая работа №4

Тема: Расширение функционала редактора с подсветкой кода до возможности запуска Python-скриптов.

Цель работы: Получить практические навыки работы с классом QProcess в среде Qt, освоить механизм запуска внешних скриптов из приложения.

Задание:

- Добавить кнопку «Выполнить» и виджет для вывода результата запуска скриптов в главное окно и скомпоновать их на форме.
- Реализовать слот для кнопки выполнения скрипта, в результате которого код программы на Python будет запущен.

Порядок выполнения:

1) Добавьте новые элементы в интерфейс по заданию. Например, редактор может выглядеть как на рисунке 2.4.1.

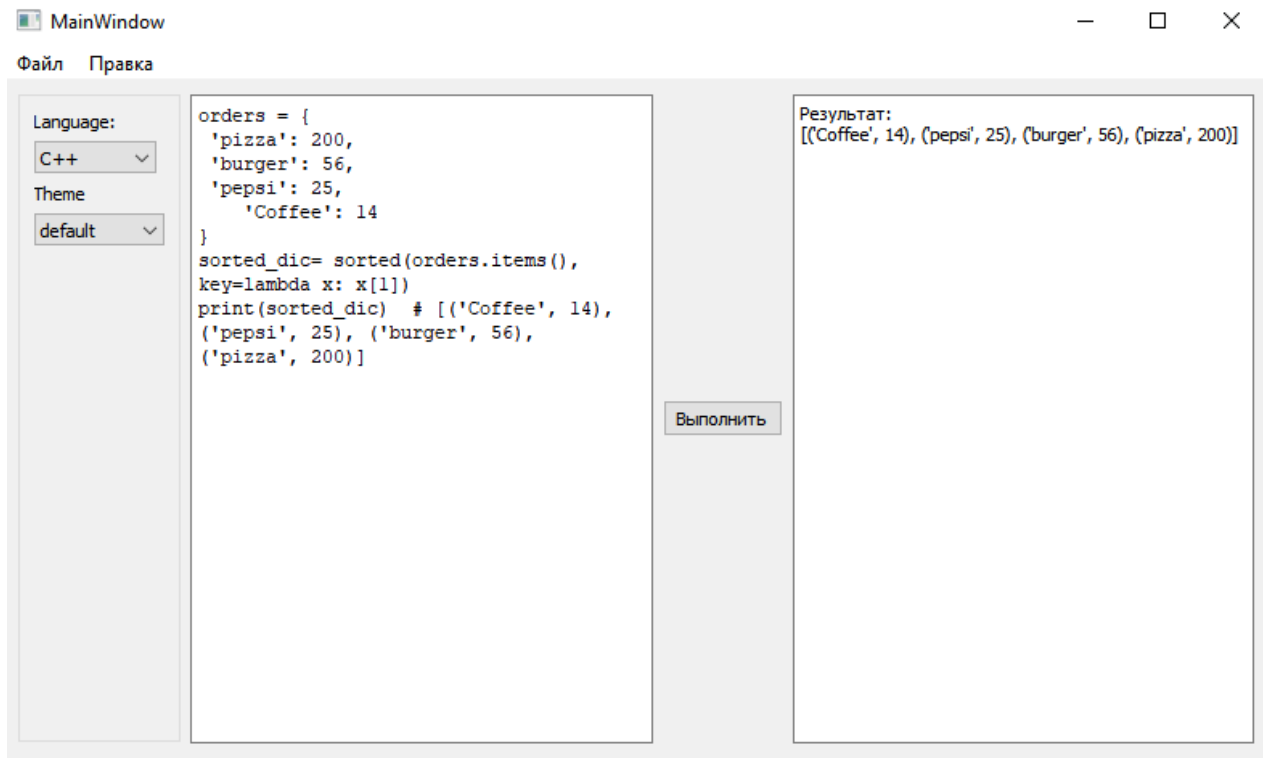


Рисунок 2.4.1. Редактор с функцией запуска скриптов

2) Для реализации запуска скриптов используйте QProcess — класс Qt, который позволяет запускать внешние программы и взаимодействовать с ними. Реализуйте слот, в котором текст программы будет сначала сохранен во временный файл, а затем этот файл будет подан на исполнение:

```
QProcess process;
process.start("python", QStringList() << filePath);
if (!process.waitForStarted()) { QMessageBox::critical(this,"Ошибка","Python не
установлен или не найден в PATH");
    return; }
process.waitForFinished(3000); // ждём до 3 секунд
// Считываем вывод и ошибки
QString output = process.readAllStandardOutput();
QString error = process.readAllStandardError();
```

Использование в данной реализации метода `waitForFinished(int msecs)` позволяет ожидать завершения процесса заданное количество миллисекунд. Этот ме-

тод возвращает true, если процесс успешно завершился и возвращает false, если истекло время ожидания или произошла ошибка.

3) Выводите результат исполнения кода в интерфейс.

4) Добавьте проверку выбранного языка из выпадающего списка, чтобы реализованный слот выполнялся только при выборе языка Python. При выборе других языков редактор должен информировать об ограниченном функционале.

5) Реализовать обработку ситуации, в которой Python-скрипт завис или выполняется слишком долго. Для этого можно использовать `process.kill()` с кнопкой «Остановить», а также реализовать проверку времени ожидания.

6) Протестируйте работу полученного приложения, проверив следующие случаи:

- Пользователь нажимает «Выполнить» с пустым редактором → приложение не должно аварийно завершаться.
- Проверить, отображается ли правильный результат исполнения простого Python-скрипта в виджете вывода.
- Если в коде есть ошибка, то она должна корректно отображаться в том же выводе.

Дополнительное задание: реализуйте возможность запуска Python-скриптов в отдельном потоке с помощью `QThread` или `QtConcurrent`, создав отдельный класс для работы с процессом.

Контрольные вопросы:

- 1) Что такое `QProcess` и зачем он используется в Qt?
- 2) Что означают параметры при вызове функции `process.start()`?
- 3) Как проверить, успешно ли запущен процесс?
- 4) Как получить вывод программы после её выполнения?
- 5) Почему важно использовать временные файлы при запуске пользовательского кода?
- 6) Как работает метод `waitForFinished()` и зачем он нужен?
- 7) Как очистить временный файл после выполнения скрипта и как можно использовать при этом `QTemporaryFile`?
- 8) Как обрабатывать ситуации, когда Python-скрипт завис или выполняется слишком долго?

2.5 Практическая работа №5

Тема: Расширение функционала MDI-приложения `QTabelo` для работы с таблицами.

Цель работы: Изучить архитектуру открытого MDI-приложения для работы с документами и расширить его функциональность, реализовав работу с таблицами.

Задание:

Склонировать репозиторий проекта QTabelo – шаблона приложения для работы с таблицами, написанного на C++ с использованием фреймворка Qt: <https://github.com/beletalabs/qtable> . Изначально это приложение включает: поддержку нескольких документов (MDI — Multiple Document Interface) во вкладках, систему меню и настраиваемую панель инструментов, работу с файлами, настройками (QSettings), а также базовую структуру документа с поддержкой нескольких табличных листов. Но реальной функциональности в приложении пока нет.

- Реализуйте создание таблиц с помощью QTableWidgetItem.
- Добавьте возможность редактирования данных таблицы пользователем, добавление и удаления строк и столбцов.

Порядок выполнения:

1) Откройте проект в Qt Creator и убедитесь, что он компилируется и запускается без ошибок. В данном проекте класс DocumentManager (наследник QMdiArea) управляет несколькими документами, а каждый документ представлен классом DocumentWindow, содержащим DocumentWidget, который в свою очередь наследуется от TableDocument, содержащий QTabWidget. То есть изначально реализована структура вкладок, но таблиц в них пока нет.

2) В методе slotAddTab(int count) класса TableDocument замените создание пустых виджетов на добавление таблицы, например так:

```
if (m_tabs->count() == 0) {
    for (int i = 0; i < count; ++i) {
        QTableWidgetItem* table = new QTableWidgetItem(this);
        table->setRowCount(15); // Стартовое количество строк
        table->setColumnCount(5); // Стартовое количество столбцов
        // Устанавливаем заголовки
        QStringList headers;
        for (int col = 0; col < 5; ++col) {
            headers << QString("Column %1").arg(col + 1);
        }
        table->setHorizontalHeaderLabels(headers);
        table->setAttribute(Qt::WA_DeleteOnClose);
        m_tabs->addTab(table, tr("Sheet %1").arg(i + 1));
    }
    m_tabs->setTabsClosable(m_tabs->count() > 1);
}
```

Здесь m_tabs является указателем на объект класса QTabWidget, который является членом класса TableDocument. Он хранит и управляет вкладками, каждая из которых содержит таблицу QTableWidgetItem. Посмотрите конструктор класса TableDocument и обратите внимание на инициализацию этого указателя.

В цикле for (int i = 0; i < count; ++i) создаётся count новых таблиц (QTableWidgetItem). Условие if (m_tabs->count() == 0) гарантирует, что таблицы создаются только при первом вызове метода (если вкладок ещё нет).

Далее установка атрибута `Qt::WA_DeleteOnClose` для таблицы обеспечит её автоматическое удаление при закрытии вкладки, а метод `addTab()` добавляет таблицы как вкладки в `m_tabs` (основной контейнер вкладок). Название вкладки генерируется автоматически на основе номера `i`.

Запустите приложение и убедитесь, что теперь при создании нового документа появляется созданная таблица (рис. 2.5.1):

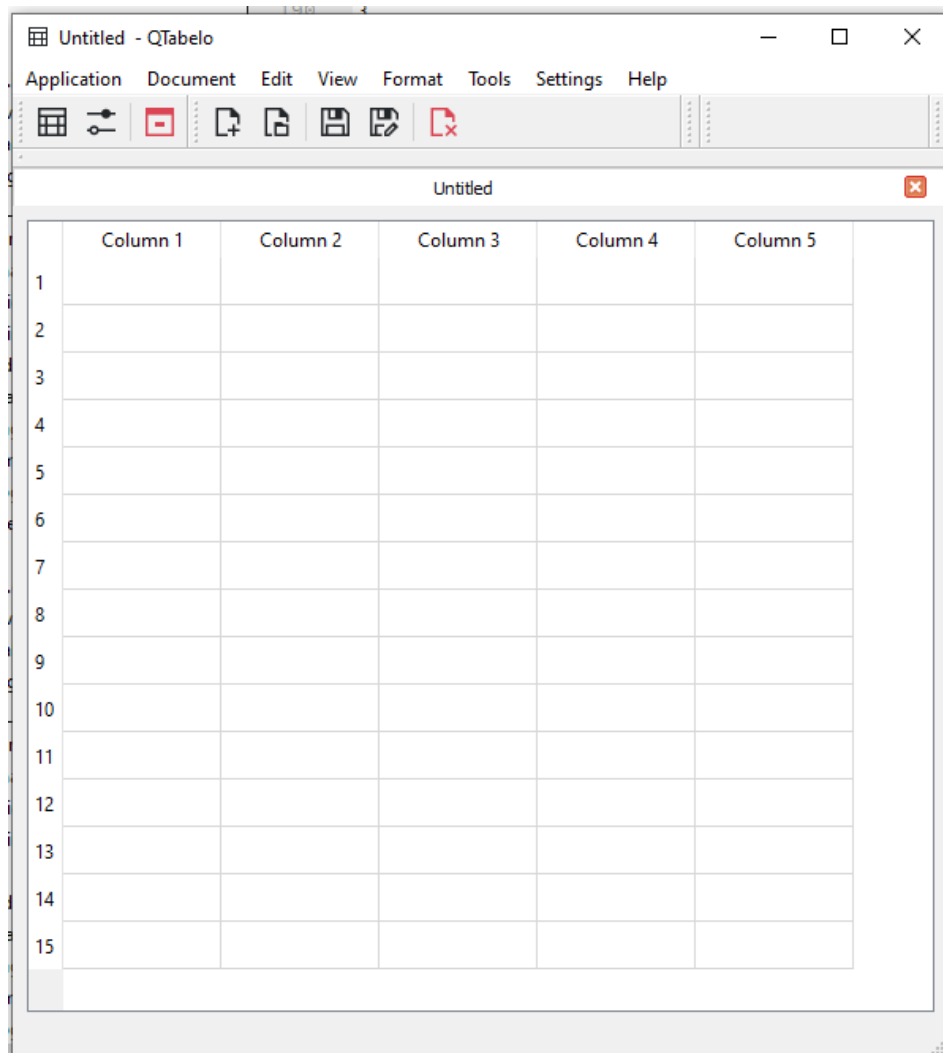


Рисунок 2.5.1. Новый документ с таблицей

3) Далее нужно реализовать возможность редактирования числа строк и столбцов таблицы. Сделать это лучше через контекстное меню, причем изменения должны влиять только на текущую таблицу. В заголовочный файл `table_document.h` добавьте слот для реализации контекстного меню:

```
void showTableContextMenu(QTableWidget *table, const QPoint &pos)
```

В этом слоте нужно будет не только создать действия, но и предусмотреть то, что когда вы добавляете строки или столбцы через контекстное меню в `TableDocument`, нужно уведомлять `DocumentWidget`, что ваш документ изме-

нился. Обратите внимание, что класс DocumentWidget имеет свойство modified (флаг изменения) и сигнал modifiedChanged(). Но так как TableDocument создаёт таблицы и не знает о свойстве modified, нужно добавить сигнал изменения содержимого, например, contentChanged (bool modified), в заголовочный файл table_document.h. Тогда реализация контекстного меню будет выглядеть так:

```
void TableDocument::showTableContextMenu(QTableWidget *table, const QPoint
&pos) {
    QMenu menu;
    QAction *actionAddRow = menu.addAction(tr("Add Row"));
    QAction *actionRemoveRow = menu.addAction(tr("Remove Last Row"));
    QAction *actionAddColumn = menu.addAction(tr("Add Column"));
    QAction *actionRemoveColumn = menu.addAction(tr("Remove Last Column"));
    QAction *selectedAction = menu.exec(table->mapToGlobal(pos));
    if (!selectedAction)
        return;
    if (selectedAction == actionAddRow) {
        table->setRowCount(table->rowCount() + 1);
        emit contentChanged(true); // Уведомляем, что документ модифицирован
    }
    else if (selectedAction == actionRemoveRow && table->rowCount() > 0) {
        table->setRowCount(table->rowCount() - 1);
        emit contentChanged(true);
    }
    else if (selectedAction == actionAddColumn) {
        int newColIndex = table->columnCount();
        table->setColumnCount(newColIndex + 1);
        table->setHorizontalHeaderItem(newColIndex, new QTableWidgetItem(
QString("Column %1").arg(newColIndex + 1)));
        emit contentChanged(true);
    }
    else if (selectedAction == actionRemoveColumn && table->columnCount() > 0)
    {
        int colCount = table->columnCount();
        table->setColumnCount(colCount - 1);
        emit contentChanged(true);
    }
}
```

Затем в конструкторе класса DocumentWidget нужно добавить эту связь между сигналом contentChanged из TableDocument и вызовом setModified для уведомления о любом изменении содержимого таблицы:

```
DocumentWidget::DocumentWidget(QWidget *parent)
: TableDocument(parent), m_modified{false}, m_url{QUrl()}
{
    setAttribute(Qt::WA_DeleteOnClose);
    connect(this, &TableDocument::contentChanged, this,
&DocumentWidget::setModified);
}
```

Здесь используется `this`, потому что `DocumentWidget` наследуется от `TableDocument`.

Теперь в слот `TableDocument::slotAddTab()` нужно добавить подключение контекстного меню:

```
void TableDocument::slotAddTab(const int count)
{
    if (m_tabs->count() == 0) {
        for (int i = 0; i < count; ++i) {
            auto *table = new QTableWidget(15, 5); // Таблица 15x5
            table->setObjectName(QStringLiteral("Sheet%1").arg(i + 1));
            // Устанавливаем заголовки
            QStringList headers;
            for (int col = 0; col < 5; ++col) {
                headers << QString("Column %1").arg(col + 1);
            }
            table->setHorizontalHeaderLabels(headers);
            table->setContextMenuPolicy(Qt::CustomContextMenu);
            // Подключаем контекстное меню
            connect(table, &QTableWidget::customContextMenuRequested,
                    this, [this, table](const QPoint &pos) {
                        showTableContextMenu(table, pos);
                    });
            // Подключаем сигнал изменения ячейки
            connect(table, &QTableWidget::itemChanged,
                    this, [this](QTableWidgetItem *) {
                        DocumentWidget *doc = qobject_cast<DocumentWidget*>(this->parent());
                        if (doc) {
                            doc->setModified(true);
                        }
                    });
            m_tabs->addTab(table, tr("Sheet %1").arg(i + 1));
        }
    }
}
```

Этот метод является ключевой частью реализации таблицы, и теперь в нем подключается контекстное меню, открывающееся по правой кнопке мыши. То есть при вызове события `customContextMenuRequested` открывается меню через метод `showTableContextMenu()`, а при изменении любой ячейки генерируется сигнал `itemChanged()`. Через `qobject_cast<DocumentWidget*>(this->parent())` получаем родительский документ и вызываем `setModified(true)`, то есть документ помечается как изменённый.

4) Протестируйте работу полученного приложения, проверив следующие случаи:

- Убедитесь, что при создании нового документа появляется таблица заданного размера с заголовками столбцов.
- Заполните ячейки данными.
- Добавьте/удалите строку и столбец.

Дополнительное задание: добавить возможность удаления всех выделенных столбцов и строк.

Контрольные вопросы:

- 1) Что такое MDI и как он реализован в Qt?
- 2) Как реализуется контекстное меню в Qt?
- 3) Какие сигналы и слоты используются для отслеживания изменений в таблице?
- 4) Как работает механизм modifiedChanged и зачем он нужен?
- 5) Как реализуется связь между TableDocument и DocumentWidget?

2.6 Практическая работа №6

Тема: Расширение функционала MDI-приложения QTabelo для работы с CSV.

Цель работы: Добавить в приложение QTabelo функциональность для импорта/экспорта табличных данных в формате CSV.

Задание:

- Реализовать загрузку данных из CSV-файла в таблицу.
- Реализовать сохранение данных таблицы в CSV-файл.
- Интегрировать новые методы в существующие слоты loadDocument() и saveDocument()

Порядок выполнения:

- 1) Реализуйте сохранение данных:

- В классе ApplicationWindow уже имеются слоты для сохранения и загрузки документа – loadDocument() и saveDocument(), но в них нет реализации. Также имеется слот importCsvToTable() для импорта табличных данных, без реализации. Нужно добавить слот exportTableToCsv() для экспорта таблиц, а также реализовать все необходимые действия для перечисленных методов.

- Реализуйте экспорт таблицы из QTableWidgetItem в CSV-файл. Для этого нужно открыть файл с переданным именем для записи, используя QFile и QTextStream, сохранить заголовки таблицы в список, затем сохранить данные строк таблицы и записать файл. Пример реализации данного слота:

```
bool ApplicationWindow::exportTableToCsv(QTableWidgetItem *table, const QString &filename)
{
    QFile file(filename);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text))
        return false;
    QTextStream out(&file);
    // Сохраняем заголовки
    QStringList headers;
    for (int col = 0; col < table->columnCount(); ++col) {
        headers << table->horizontalHeaderItem(col)->text();
    }
}
```

```

    out << headers.join(",") + "\n";
    // Сохраняем строки
    for (int row = 0; row < table->rowCount(); ++row) {
        QStringList rowText;
        for (int col = 0; col < table->columnCount(); ++col) {
            QTableWidgetItem *item = table->item(row, col);
            rowText << (item ? item->text() : "");
        }
        out << rowText.join(",") + "\n";
    }
    file.close();
    return true;
}

```

- Обновите слот `saveDocument()` в `application_window.cpp`. В нем можно добавить поиск активной таблицы через иерархию виджетов, используя метод `findChild()`, а также необходим вызов реализованного слота для экспорта таблицы. При успешном сохранении вызывайте метод `resetModified()`, который сбрасывает флаг «измененности» документа и указывает, что все изменения сохранены. Этот метод в проекте `QTabelo` играет ключевую роль в управлении состоянием документа и вызывается при создании пустого документа, после успешного сохранения и при открытии нового документа. Пример реализации слота `saveDocument()`:

```

bool ApplicationWindow::saveDocument(DocumentWidget *document, const QUrl
&altUrl)
{
    Q_UNUSED(resetModified());
    // Определяем URL для сохранения
    QUrl url = altUrl.isEmpty() ? document->url() : altUrl;
    if (!url.isValid()) {
        QMessageBox::critical(this, tr("Error"), tr("Invalid file location."));
        return false;
    }
    // Получаем текущую таблицу через безопасный метод
    QTableWidgetItem *table = nullptr;
    if (auto tabs = document->findChild<QTabWidget*>()) {
        table = qobject_cast<QTableWidgetItem*>(tabs->currentWidget());
    }
    if (!table) {
        QMessageBox::critical(this, tr("Error"), tr("No table data to save."));
        return false;
    }
    // Экспортируем данные
    if (!exportTableToCsv(table, url.toLocalFile())) {
        QMessageBox::critical(this, tr("Error"), tr("Could not save the
file."));
        return false;
    }
    document->resetModified();
    document->setUrl(url);
}

```

```

    m_recentDocuments->addUrl(url);
    return true;
}

```

5) Реализуйте импорт таблицы из CSV-файла в соответствующем слоте `importCsvToTable()` файла `application_window.cpp`. Для этого откройте файл для чтения, пользуясь `QFile` и `QTextStream`. Чтобы загрузить новую структуру таблицы, сначала вызываются методы очистки ячеек, строк и столбцов. Затем происходит построчное считывание данных и результат записывается в список строк, где первая строка содержит заголовки столбцов, а последующие строки – данные. Пример реализации слота для импорта таблицы:

```

bool ApplicationWindow::importCsvToTable(const QString &filename, QTableWidgetItem *table)
{
    QFile file(filename);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        QMessageBox::critical(this, tr("Error"), tr("Could not open the file."));
        return false;
    }
    QTextStream in(&file);
    table->clearContents();
    table->setRowCount(0);
    table->setColumnCount(0);
    int row = 0;
    while (!in.atEnd()) {
        QString line = in.readLine();
        QStringList fields = line.split(',');
        if (row == 0) {
            // Устанавливаем заголовки
            table->setColumnCount(fields.size());
            table->setHorizontalHeaderLabels(fields);
        } else {
            table->insertRow(table->rowCount());
            for (int col = 0; col < fields.size(); ++col) {
                if (col >= table->columnCount())
                    table->insertColumn(table->columnCount());
                table->setItem(row - 1, col, new QTableWidgetItem(fields[col]));
            }
            ++row;
        }
    }
    file.close();
    return true;
}

```

- Обновите слот `loadDocument()` в `application_window.cpp`. В нем сначала создается новый документ, затем с помощью встроенного механизма `documentCountChanged()` создается новая вкладка и инициализируется виджет таблицы.

Затем происходит поиск таблицы в иерархии виджетов, аналогично как было реализовано при сохранении, и вызывается метод импорта данных из CSV. Пример реализации слота loadDocument():

```
bool ApplicationWindow::loadDocument(const QUrl &url)
{
    DocumentWidget *document = createDocument();
    if (!document) return false;
    // Добавляем таблицу через стандартный механизм
    document->documentCountChanged(1); // Используем существующий метод
    // Получаем таблицу через доступ к m_tabs
    QTableWidget *table = nullptr;
    if (document->findChild<QTabWidget*>()) { // Проверяем наличие QTabWidget
        QTabWidget *tabs = document->findChild<QTabWidget*>();
        table = qobject_cast<QTableWidget*>(tabs->currentWidget());
    }
    if (!table) {
        document->close();
        return false;
    }
    // Загружаем данные
    if (!importCsvToTable(url.toLocalFile(), table)) {
        document->close();
        return false;
    }
    document->show();
    document->setUrl(url);
    m_recentDocuments->addUrl(url);
    documentCreated();
    return true;
}
```

6) Протестируйте приложение, следуя плану:

- Создайте таблицу, заполните её данными и сохраните как CSV-файл.
- Закройте текущую таблицу.
- Проверьте загрузку через меню "Файл → Открыть".
- Измените данные в таблице.
- Сохраните таблицу через "Файл → Сохранить".
- Проверьте содержимое сохраненного файла.

Дополнительное задание: Перевести QTableWidget на QTableView с кастомной моделью, унаследованной от QAbstractTableModel.

Контрольные вопросы:

- 1) Какие этапы включает загрузка данных из CSV в QTableWidget?
- 2) Какие классы в Qt отвечают за управление документами?
- 3) Как связаны между собой классы ApplicationWindow, DocumentManager и DocumentWidget при работе с CSV?
- 4) Какие сигналы и слоты задействованы при импорте/экспорте таблиц?
- 5) Какие улучшения можно добавить в функциональность работы с CSV?

2.7 Практическая работа №7

Тема: Добавление генератора диаграмм в приложение QTabelo.

Цель работы: научиться использовать QGraphicsScene, QMap, а также классы для работы с 2D графикой в Qt и добавить в приложение QTabelo преобразование выделенных данных таблицы в диаграмму.

Задание:

- Реализовать визуализацию данных без копирования в Excel.
- Построить интерактивные диаграммы внутри QTabelo.
- Сохранять диаграммы как изображения.

Порядок выполнения:

1) Создайте новый класс, например ChartBuilder, для создания столбчатых диаграмм на основе выделенных данных из таблицы. - Визуализация данных должна происходить с использованием графической сцены, поэтому класс ChartBuilder должен быть унаследован от QGraphicsScene. В конструктор необходимо передавать таблицу, из которой берутся данные:

```
explicit ChartBuilder(QTableWidget *sourceTable, QObject *parent = nullptr);
```

- Добавьте в класс метод для построения диаграммы, в который будут передаваться выделенные ячейки:

```
void buildBarChart(const QModelIndexList &selectedCells);
```

- Добавьте в класс ChartBuilder метод для сохранения диаграммы в PNG файл, например: void exportToPng(const QString &path);

- Приватные поля в классе ChartBuilder – это указатель на исходную таблицу с данными (например, QTableWidget *m_sourceTable) и контейнер QMap<QString, double>, например, m_data для хранения данных диаграммы.

2) Реализуйте логику диаграмм в методе buildBarChart(), следуя рекомендациям:

- Сначала метод удаляет все предыдущие элементы с графической сцены с помощью clear().

- Для сбора данных метод проходит по переданному ему списку выделенных ячеек, обрабатывая их парами (четные индексы - подписи, нечетные - значения). Данные сохраняются в контейнер QMap<QString, double>, определенным в классе ChartBuilder.

- Для подготовки к визуализации устанавливаются постоянные параметры: ширина столбцов и максимальная высота диаграммы.

- Находится максимальное значение в данных для пропорционального отображения всех столбцов.

- Отрисовка диаграммы происходит следующим образом. Для каждой пары «подпись-значение» создается столбец (QGraphicsRectItem) и заливается цветом. Затем добавляется текстовая подпись (QGraphicsTextItem) с названием и значением. И далее элементы добавляются на сцену методом addItem().

- Предусмотреть, что каждый новый столбец рисуется с отступом от предыдущего.

- Итоговый метод должен автоматически масштабировать диаграмму, обеспечивая наглядное представление данных независимо от их абсолютных значений.

Добавьте в контекстное меню таблицы вызов мастера диаграмм. Для этого в методе `TableDocument::showTableContextMenu()` необходимо добавить новое действие `QAction` и подключить его к имеющемуся меню. Действия для создания и построения диаграммы можно реализовать так:

```
auto *chart = new ChartBuilder(table);
chart->buildBarChart(table->selectionModel()->selectedIndexes());
QGraphicsView *chartView = new QGraphicsView(chart);
int tabIndex = m_tabs->addTab(chartView, tr("Диаграмма"));
m_tabs->setCurrentIndex(tabIndex);
emit contentChanged(true);
```

3) Добавьте контекстное меню к диаграммам. Для этого можно переопределить метод `contextMenuEvent()` в классе `ChartBuilder` (так как он наследуется от `GraphicsScene`) и реализовать в нем действие для вызова слота сохранения диаграммы. Метод `contextMenuEvent()` будет обрабатывать клик правой кнопкой мыши по сцене и показывать контекстное меню с опцией экспорта в PNG.

4) Протестируйте работу приложения, следуя плану:

- Проверить поведение при передаче пустого `QModelIndexList` (не должно быть ошибок, сцена должна остаться пустой).
- Проверить обработку, когда количество выделенных ячеек нечетное (последняя ячейка должна игнорироваться).
- Проверить реакцию на текст вместо чисел в ячейках значений, пустые ячейки, специальные символы в подписях.
- Проверить корректное отображение столбцов: соответствие высоты столбцов переданным значениям и правильное позиционирование столбцов.
- Подписи данных должны корректно отображать информацию и располагаться под столбцами.
- Проверить, что повторный вызов с новыми данными полностью перерисовывает диаграмму.

Дополнительные задания:

1. Реализовать функцию обновления диаграммы при изменении данных в таблице.
2. Реализовать элементы управления для настройки диаграммы.

Контрольные вопросы:

1. Как связаны `ChartBuilder` и `QGraphicsView`?
2. Как сохранить диаграмму в изображение? Какие методы для этого используются?
3. Как обрабатывается контекстное меню (`contextMenuEvent`) на сцене?
4. Почему для хранения данных используется `QMap<QString, double>`?

5. Что происходит при многократном вызове `buildBarChart` с разными данными?
6. Как очищается сцена перед построением новой диаграммы?
7. Какие классы Qt используются для создания столбцов и подписей?
8. Как рассчитывается высота столбцов диаграммы?
9. Как обрабатываются очень длинные подписи (перенос, обрезание)?
10. Как позиционируются столбцы и подписи на сцене?

3. МЕТОДИЧЕСКИЕ УКАЗАНИЯ К КУРСОВОЙ РАБОТЕ

3.1 Задание

Цель работы: Разработать десктопное многооконное приложение на языке C++ в среде Qt Creator, реализующее функционал предметной области по варианту. Распределение вариантов осуществляется преподавателем **до первого контрольного срока**. Далее вся работа по курсовому проекту проходит в рамках назначенной темы.

Требования: Приложение должно содержать минимум 5 классов, отражающих сущности предметной области, поддерживать работу с несколькими пользователями. Обязательно использование минимум двух паттернов проектирования из следующих: Observer, Strategy, Factory, Command, Composite. Данные приложения (пользователи, сущности, настройки) должны сохраняться локально (JSON, XML, или SQLite).

Варианты:

1) Редактор тестов для проверки знаний по предмету (2 чел.)

- ✓ Создание, редактирование и удаление тестов и вопросов.
- ✓ Поддержка типов вопросов: одиночный выбор, множественный выбор, открытый ответ.
- ✓ Привязка тестов к предметам и темам.
- ✓ Прохождение теста с автоматической проверкой и подсчётом баллов.
- ✓ Поиск по предмету, типу вопроса, сложности.
- ✓ Сохранение результатов, импорт/экспорт тестов.
- ✓ Визуализация статистики (рейтинг, прогресс).

Рекомендуется использовать следующие сущности:

- User — студент/преподаватель (разный интерфейс)
- Test — тест (название, предмет, тема)
- Question — вопрос (текст, тип, сложность)
- Answer — ответ (правильный/неправильный, текст)
- TestSession — сессия прохождения, результаты

Рекомендуемые паттерны:

- Factory — создание разных типов вопросов.
- Observer — обновление прогресса и UI при изменении состояния.

2) Организация работы школьной столовой (2 чел.)

- ✓ Управление меню и ценами.
- ✓ Оформление заказов с автоматическим списанием средств.
- ✓ Поиск блюд по категории, цене.
- ✓ Фильтрация заказов по дате, ученику.
- ✓ Отчёты: выручка, популярные блюда.
- ✓ Импорт/экспорт меню, сохранение при выходе.

Рекомендуется использовать следующие сущности:

- User — администратор/ученик (разный интерфейс)

- Meal — блюдо (название, цена, категория)
- Order — заказ (дата, блюда)
- Category — завтрак, обед, перекус
- ReportManager — формирование отчётов

Рекомендуемые паттерны:

- Strategy — разные стратегии сортировки и формирования отчётов.
- Observer — обновление баланса ученика при оформлении заказа.

3) Создание комментируемых электронных фотоальбомов (1 чел.)

- ✓ Добавление, редактирование, удаление фото.
- ✓ Организация по альбомам и папкам.
- ✓ Поиск по тегам, дате, описанию.
- ✓ Импорт/экспорт альбома (со структурой).
- ✓ Визуализация хронологии (лента).

Рекомендуется использовать следующие сущности:

- User — владелец альбома
- Photo — изображение (файл, описание, дата)
- Album — альбом (название, теги)
- Tag — метка (например, "отпуск", "семья")
- PhotoManager — загрузка, экспорт, поиск

Рекомендуемые паттерны:

- Composite — вложенные альбомы в виде дерева.
- Strategy — разные стратегии поиска (по тегу, по дате).

4) Работа ремонтной мастерской техники (2 чел.)

- ✓ Оформление заказа клиентом.
- ✓ Приём заказа, смена статуса "принят", "в работе", "готов", "выдан").
- ✓ Учёт и списание запчастей при ремонте.
- ✓ Поиск заказов по клиенту, статусу, дате.
- ✓ Формирование счёта и истории заказов.
- ✓ Экспорт истории заказов.

Рекомендуется использовать следующие сущности:

- User — менеджер/клиент (разный интерфейс)
- RepairOrder — заказ (устройство, неисправность)
- Device — тип устройства (ноутбук, телефон)
- Part — запчасть (артикул, количество)
- StatusTracker — отслеживание статуса

Рекомендуемые паттерны:

- Strategy — смена поведения по статусу.
- Observer — уведомления о смене статуса.

5) Планировщик задач по разным темам с уведомлениями (1 чел.)

- ✓ Создание, редактирование, выполнение задач.
- ✓ Группировка по проектам, категориям.
- ✓ Напоминания за заданное время до дедлайна.
- ✓ Поиск по приоритету, дате, проекту.

✓ Импорт/экспорт списка задач.

Рекомендуется использовать следующие сущности:

- User — владелец задач
- Task — задача (название, дата, приоритет)
- Project — проект или категория
- Reminder — напоминание (время до дедлайна)
- TaskManager — сортировка, фильтрация

Рекомендуемые паттерны:

- Command — отмена/повтор действия.
- Observer — автоматическое обновление при наступлении напоминания.

6) Обучающее приложение для детей дошкольного возраста (2 чел.)

- ✓ Прохождение интерактивных заданий.
- ✓ Система баллов и уровней сложности.
- ✓ Призы и поощрения за выполнение.
- ✓ Поиск заданий по типу (цвета, формы).
- ✓ Визуализация прогресса (прогресс-бар, диаграммы).
- ✓ Импорт/экспорт заданий и профиля ребенка с прогрессом.

Рекомендуется использовать следующие сущности:

- User — ребёнок и родитель/куратор (разный интерфейс)
- Exercise — задание (буквы, цифры, фигуры)
- Reward — награда (баллы, значки)
- Level — уровень сложности
- ProgressTracker — статистика выполнения

Рекомендуемые паттерны:

- Strategy — выбор типа задания.
- Observer — обновление прогресса после выполнения.

7) Система ведения личного дневника с тегами и поиском (1 чел.)

- ✓ Добавление, редактирование, удаление записей.
- ✓ Поиск по тегам, дате, тексту.
- ✓ Фильтрация по месяцу, настроению.
- ✓ Визуализация активности (календарь записей).
- ✓ Экспорт и импорт дневника.

Рекомендуется использовать следующие сущности:

- User — автор записей
- Entry — запись (текст, дата, теги)
- Tag — метка (настроение, тема)
- SearchManager — поиск по ключевым словам
- ExportManager — импорт/экспорт

Рекомендуемые паттерны:

- Strategy — разные стратегии поиска (по тегу, по дате).
- Command — отмена удаления записи.

8) Организация работы магазина бытовых товаров (3 чел.)

- ✓ Учёт товаров: артикул, название, цена, остаток.
- ✓ Покупка товаров, заказ.
- ✓ Оформление продаж и поставок товаров.
- ✓ Поиск по артикулу, названию, категории.
- ✓ Отчёты: прибыль, популярные товары.
- ✓ Импорт/экспорт данных о товарах и операциях.

Рекомендуется использовать следующие сущности:

- User — кассир, администратор, клиент (разные интерфейсы)
- Product — товар (артикул, цена, остаток)
- Sale — продажа (чек, состав)
- Supply — поставка товаров
- ReportManager — прибыль, топ-товары

Рекомендуемые паттерны:

- Factory — создание разных типов операций (продажа/поставка).
- Observer — обновление остатков в реальном времени.

9) Учет персональной библиотеки книг (1 чел.)

- ✓ Добавление, редактирование книг.
- ✓ Поиск по автору, названию, жанру.
- ✓ Сортировка по алфавиту, году.
- ✓ Формирование списков «Прочитанные», «Планирую прочитать».
- ✓ Импорт/экспорт библиотеки.

Рекомендуется использовать следующие сущности:

- User — владелец библиотеки
- Book — книга (автор, название, жанр)
- Status — статус (читаю, прочитана)
- Tag — метки (например, "фантастика")
- ListManager — списки "Прочитанные", "Хочу прочитать".

Рекомендуемые паттерны:

- Strategy — разные способы сортировки.
- Composite — списки книг как коллекции.

10) Приложение для трекинга полезных привычек (1 чел.)

- ✓ Создание привычек с расписанием.
- ✓ Отметка выполнения за день.
- ✓ Визуализация прогресса (стрики, календарь)..
- ✓ Напоминания.
- ✓ Поиск по названию, частоте.

Рекомендуется использовать следующие сущности:

- User — пользователь
- Habit — привычка (название, периодичность)
- Record — отметка выполнения (дата)
- CalendarView — календарь выполнений
- StreakManager — подсчёт "серий"

Рекомендуемые паттерны:

- Observer — обновление календаря при изменении.
- Strategy — разные стратегии визуализации (тепловая карта, график).

11) Редактор целей и достижений в разных сферах жизни (1 чел.)

- ✓ Создание целей с подцелями.
- ✓ Отслеживание прогресса.
- ✓ Напоминания о сроках.
- ✓ Поиск по сроку, сфере (работа, здоровье).
- ✓ Экспорт данных в файл.

Рекомендуется использовать следующие сущности:

- User — владелец целей
- Goal — цель (название, срок)
- Subgoal — подцель
- Progress — прогресс (в %)
- Reminder — напоминание о дедлайне

Рекомендуемые паттерны:

- Composite — цели и подцели как дерево.
- Observer — обновление прогресса при изменении подцелей.

12) Система учета домашних рецептов (1 чел.)

- ✓ Добавление рецептов: название, фото, ингредиенты, шаги приготовления, время.
- ✓ Поиск по ингредиентам, времени приготовления.
- ✓ Фильтрация по типу блюда (первое, десерт и т.д.).
- ✓ Возможность пометки «избранное».
- ✓ Импорт/экспорт списка рецептов.

Рекомендуется использовать следующие сущности:

- User — повар
- Recipe — рецепт (название, ингредиенты, шаги)
- Ingredient — ингредиент
- Category — тип блюда (первое, десерт)
- FavouritesManager — избранное

Рекомендуемые паттерны:

- Strategy — разные фильтры.
- Factory — создание рецепта по шаблону.

13) Приложение для изучения музыкальных аккордов и гамм (2 чел.)

- ✓ Создание и редактирование базы аккордов и гамм для гитары и фортепиано.
- ✓ Просмотр схем аккордов и гамм с возможностью масштабирования и поворота (для гитары — вид схемы ладов, для фортепиано — клавиатура).
- ✓ Поиск по названию, тональности, типу (мажор, минор), инструменту.

- ✓ Фильтрация по сложности (начинающий, средний, продвинутый).
- ✓ Режимы обучения: «Угадай аккорд по схеме», «Найди схему по названию».
- ✓ Возможность создавать кастомные наборы (например, "Аккорды для блюза", "Минорные гаммы") — функция для учителя.
- ✓ Импорт/экспорт базы данных (в JSON) — для обмена наборами между пользователями.
- ✓ Сохранение прогресса при выходе, загрузка при запуске.

Рекомендуется использовать следующие сущности:

- User — учитель (автор)/ученик (разный интерфейс)
- Chord — аккорд (название, инструмент, схема)
- Scale — гамма
- Instrument — тип (гитара, фортепиано)
- TrainingMode — режим "угадай по схеме"

Рекомендуемые паттерны:

- Factory — создание схемы под инструмент.
- Strategy — выбор режима обучения.

14) Работа компьютерного клуба (2 чел.)

- ✓ Регистрация клиента.
- ✓ Выбор времени посещения и запись клиента.
- ✓ Подтверждение записи клиента администратором.
- ✓ Учет времени сессии.
- ✓ Расчет стоимости по тарифу.
- ✓ Поиск по клиенту, дате, ПК.
- ✓ Отчёты: доход, загруженность.
- ✓ Сохранение истории.

Рекомендуется использовать следующие сущности:

- User — клиент/администратор (разный интерфейс)
- Session — сессия (время входа/выхода)
- Rate — тариф (руб/час)
- Computer — номер ПК
- ReportManager — доход, загруженность

Рекомендуемые паттерны:

- Strategy — разные тарифы.
- Observer — обновление стоимости в реальном времени.

15) Приложение для составления расписания дополнительных занятий детей (2 чел.)

- ✓ Создание расписания для каждого ребенка.
- ✓ Повторяющиеся занятия с автозаполнением.
- ✓ Поиск по дню, учителю, предмету.
- ✓ Фильтрация по направлению развития ребенка.
- ✓ Импорт/экспорт расписания.
- ✓ Визуализация по дням недели, календарь.

✓ Напоминания о занятиях для родителя.

Рекомендуется использовать следующие сущности:

- User — родитель/ребенок (разный интерфейс)
- Lesson — занятие (название, время, место, педагог)
- Schedule — расписание по дням
- Teacher — учитель
- ExportManager — экспорт расписания

Рекомендуемые паттерны:

- Composite — расписание как коллекция дней
- Command — отмена изменений

16) Организация работы агентства по сдаче автомобилей в аренду (2 чел.)

✓ Учет автомобилей: марка, модель, статус.

✓ Заказ аренды.

✓ Оформление аренды: клиент, даты, стоимость.

✓ Расчет по дням, штрафы за просрочку.

✓ Поиск по марке, дате, клиенту.

✓ Отчёты по доходу.

✓ Импорт/экспорт данных.

Рекомендуется использовать следующие сущности:

- User — клиент/администратор (разный интерфейс)
- Car — автомобиль (марка, модель, статус)
- Rental — аренда (даты, стоимость)
- Fine — штраф за просрочку
- ReportManager — доход, загруженность

Рекомендуемые паттерны:

- Strategy — расчёт стоимости (день/неделя/штраф)
- Observer — обновление статуса авто при возврате

17) Адаптивный комплекс тренировок для мозга (2 чел.)

✓ Несколько типов мини-игр: память, внимание, логика.

✓ Уровни сложности в зависимости от набранных баллов.

✓ Поиск по типу игры.

✓ Визуализация прогресса по разным направлениям тренировок.

✓ Импорт/экспорт статистики за указанный период.

Рекомендуется использовать следующие сущности:

- User — игрок
- Game — тип игры (память, логика)
- Level — уровень сложности
- Result — результат прохождения
- ProgressTracker — статистика

Рекомендуемые паттерны:

- Factory — создание игры по типу
- Strategy — выбор алгоритма сложности на основе прогресса.

18) Создание и симуляция жизни фантастических питомцев (2 чел.)

- ✓ Создание питомца по критериям (скорость изменения потребностей, внешний вид).
- ✓ Выбор комнаты для питомца.
- ✓ Управление несколькими питомцами.
- ✓ Потребности питомца: голод, сон, настроение, здоровье.
- ✓ Действия: кормить, играть, спать.
- ✓ Монеты за игры для покупки еды и предметов в комнату.
- ✓ Прогресс возраста со сменой аватара с течением времени.
- ✓ Сохранение состояния игры.

Рекомендуется использовать следующие сущности:

- User — владелец
- Pet — питомец (голод, сон, настроение)
- Action — действие (кормить, играть)
- Need — потребность
- SaveManager — сохранение состояния

Рекомендуемые паттерны:

- Observer — обновление состояния при действии
- Command — отмена действия

19) Пополняемая энциклопедия полезных лесных трав (2 чел.)

- ✓ Добавление, редактирование растений.
- ✓ Поиск и фильтрация по сезону, типу.
- ✓ Просмотр и добавление фото и описаний.
- ✓ Список избранного для пользователя.
- ✓ Импорт/экспорт базы.
- ✓ Визуализация по сезонам.

Рекомендуется использовать следующие сущности:

- User — ботаник/пользователь (разный интерфейс)
- Plant — растение (название, фото, описание)
- Family — семейство
- Season — сезон сбора
- SearchManager — поиск по названию, семейству

Рекомендуемые паттерны:

- Strategy — разные стратегии фильтрации.
- Composite — группы растений по сезону

20) Учет и контроль работы в туристическом агентстве (2 чел.)

- ✓ Управление турами: направление, даты, стоимость, вместимость.
- ✓ Бронирование тура клиентом с подтверждением.
- ✓ Оформление оплаты и отслеживание статуса.
- ✓ Поиск туров по направлению, дате, цене.
- ✓ Фильтрация бронирований по клиенту, статусу.
- ✓ Формирование отчетов: доход, популярные направления.

- ✓ Импорт/экспорт данных о турах и клиентах.
- ✓ Сохранение состояния при выходе.

Рекомендуется использовать следующие сущности:

- User — клиент / менеджер агентства (разный интерфейс)
- Tour — тур (направление, даты, стоимость, количество мест)
- Booking — бронирование (клиент, тур, статус оплаты)
- Payment — оплата (сумма, дата, статус)
- ReportManager — формирование отчётов

Рекомендуемые паттерны:

- Strategy — выбор стратегии расчёта стоимости (горящий тур, раннее бронирование).
- Observer — уведомление о заполнении тура или изменении статуса брони.

21) Адаптивный дневник питания и тренировок (1 чел.)

- ✓ Учёт приёмов пищи: продукты, порции, калории.
- ✓ Учёт тренировок: тип, длительность, сожжённые калории.
- ✓ Подсчёт суточного баланса калорий.
- ✓ Анализ прогресса: графики веса, активности, питания.
- ✓ Поиск записей по дате, типу (еда/тренировка).
- ✓ Фильтрация по цели (похудение, набор массы).
- ✓ Рекомендации на основе данных (упрощённые, по шаблонам).
- ✓ Импорт/экспорт дневника.

Рекомендуется использовать следующие сущности:

- User — пользователь (цель, вес, рост)
- Meal — приём пищи (продукты, калории)
- Workout — тренировка (тип, время, нагрузка)
- CalorieCounter — подсчёт калорий
- ProgressTracker — визуализация прогресса

Рекомендуемые паттерны:

- Strategy — выбор стратегии подсчёта нормы калорий (по полу, возрасту).
- Observer — обновление прогресса при добавлении записи.

22) Учет заказов и расходных материалов швейного ателье (2 чел.)

- ✓ Учёт заказов: клиент, изделие, срок выполнения, стоимость.
- ✓ Учёт тканей и фурнитуры: остатки, типы материалов.
- ✓ Списание материалов при выполнении заказа.
- ✓ Поиск заказов по клиенту, сроку, статусу.
- ✓ Фильтрация по типу изделия (платье, костюм).
- ✓ Отчёты: расход материалов, прибыль по заказам.
- ✓ Импорт/экспорт данных об остатках и заказах.

Рекомендуется использовать следующие сущности:

- User — закройщик / администратор (разный интерфейс)
- Order — заказ (клиент, изделие, срок, стоимость)

- Material — ткань или фурнитура (название, остаток)
- Supply — поставка материалов
- ReportManager — отчёты по расходам и прибыли

Рекомендуемые паттерны:

- Strategy — выбор стратегии расчёта стоимости пошивки.
- Observer — обновление остатков при списании материалов.

23) Организация работы свадебного агентства (2 чел.)

- ✓ Учёт заказов: клиенты, дата свадьбы, бюджет.
- ✓ Назначение услуг: фотограф, ведущий, декор, транспорт.
- ✓ Ведение чек-листа задач (до, во время, после свадьбы).
- ✓ Отслеживание оплат по частям.
- ✓ Поиск по дате, клиенту, статусу оплаты.
- ✓ Фильтрация по бюджету, типу услуги.
- ✓ Импорт/экспорт чек-листов и заказов.

Рекомендуется использовать следующие сущности:

- User — клиент / организатор (разный интерфейс)
- WeddingOrder — заказ свадьбы (дата, бюджет)
- Service — услуга (фото, ведущий, декор)
- TaskList — чек-лист задач
- PaymentTracker — отслеживание оплат

Рекомендуемые паттерны:

- Composite — чек-лист как иерархия задач.
- Observer — обновление статуса заказа при завершении задач.

24) Администрирование фитнес клуба (2 чел.)

- ✓ Учёт абонементов: тип (разовый, месячный), срок, стоимость.
- ✓ Отметка посещений клиентов.
- ✓ Напоминания о завершении абонемента.
- ✓ Поиск по клиенту, типу абонемента, дате.
- ✓ Фильтрация по статусу (активен, истёк).
- ✓ Отчёты: посещаемость, доход.
- ✓ Импорт/экспорт данных о клиентах и абонементы.

Рекомендуется использовать следующие сущности:

- User — клиент / администратор (разный интерфейс)
- Membership — абонемент (тип, срок, стоимость)
- Visit — посещение (дата, время)
- Trainer — тренер (опционально)
- ReportManager — отчёты по посещаемости

Рекомендуемые паттерны:

- Strategy — выбор стратегии начисления (разовый, групповой).
- Observer — уведомление о завершении абонемента.

25) Редактируемый календарь садовода (1 чел.)

- ✓ Планирование работ: посадка, полив, обрезка, удобрение.

- ✓ Привязка к культурам (огурцы, помидоры) и сезонам.
- ✓ Добавление напоминаний о предстоящих работах.
- ✓ Цветовая маркировка по типу работ.
- ✓ Поиск по культуре, типу работы, дате.
- ✓ Фильтрация по сезону (весна, лето).
- ✓ Импорт/экспорт календаря.
- ✓ Сохранение при выходе.

Рекомендуется использовать следующие сущности:

- User — садовод
- GardenTask — задача (посадка, полив)
- Culture — культура (томат, картофель)
- Season — сезон (весна, лето)
- Reminder — напоминание о работе

Рекомендуемые паттерны:

- Strategy — выбор цветовой схемы по типу работы.
- Observer — обновление календаря при добавлении задачи.

26) Работа сервиса аренды техники (2 чел.)

- ✓ Учёт инвентаря: тип (дрель, генератор), статус, стоимость аренды.
- ✓ Оформление аренды: клиент, срок, залог.
- ✓ Расчёт стоимости по дням и типу техники.
- ✓ Поиск по типу техники, клиенту, дате.
- ✓ Фильтрация по статусу (в аренде, свободен).
- ✓ Отчёты: доход, загруженность инвентаря.
- ✓ Импорт/экспорт данных об оборудовании.

Рекомендуется использовать следующие сущности:

- User — клиент / администратор (разный интерфейс)
- Equipment — единица техники (тип, статус, цена в сутки)
- Rental — аренда (клиент, даты, залог)
- Rate — тариф (по типу техники)
- ReportManager — отчёты по доходу и использованию

Рекомендуемые паттерны:

- Strategy — расчёт стоимости (день/неделя/сезонная скидка).
- Observer — обновление статуса техники при оформлении аренды.

27) Создание собственного журнала комиксов (1 чел.)

- ✓ Создание страниц: панели, текст, изображения.
- ✓ Использование шаблонов разметки.
- ✓ Добавление текста и изображений в панели.
- ✓ Сохранение и загрузка журнала.
- ✓ Поиск по названию страницы, дате.
- ✓ Фильтрация по статусу (черновик, готово).
- ✓ Импорт/экспорт журнала (с сохранением структуры).

Рекомендуется использовать следующие сущности:

- User — автор комикса

- ComicPage — страница комикса (панели, текст)
- Panel — панель (изображение, текст)
- Template — шаблон разметки
- SaveManager — сохранение и загрузка проекта

Рекомендуемые паттерны:

- Composite — страница как коллекция панелей.
- Command — отмена/повтор действия (например, удаление панели).

28) Учет занятий спортивной школы для разных направлений (3 чел.)

- ✓ Создание и редактирование групп администратором: направление (футбол, танцы, плавание), тренер, расписание (дни, время, зал).
- ✓ Добавление/удаление тренеров, учеников.
- ✓ Учёт посещаемости учеников тренерами.
- ✓ Просмотр расписания своих групп тренером.
- ✓ Поиск по ученику, группе, дате.
- ✓ Просмотр своего расписания и прогресса учениками.
- ✓ Фильтрация по направлению, тренеру.
- ✓ Отчёты: посещаемость, прогресс учеников.
- ✓ Импорт/экспорт расписания и посещаемости.

Рекомендуется использовать следующие сущности:

- User — ученик / тренер / администратор (разные интерфейсы)
- Group — группа (направление, тренер, расписание)
- Attendance — посещаемость (ученик, дата, статус)
- ScheduleManager — управление расписанием, проверка конфликтов
- Trainer — тренер (ФИО, направление)
- ReportManager — отчёты по посещаемости

Рекомендуемые паттерны:

- Strategy — выбор стратегии подсчёта прогресса (по посещениям).
- Observer — обновление статуса ученика при отметке посещения.

29) Альбом фотографий и достижений ребенка от 0 до 14 лет (1 чел.)

- ✓ Добавление фото, описания, даты события.
- ✓ Организация по возрастным этапам (0–1, 1–3, 3–7 и т.д.).
- ✓ Хронология развития: рост, речь, навыки.
- ✓ Поиск по возрасту, событию, дате.
- ✓ Фильтрация по типу события (день рождения, первый шаг).
- ✓ Импорт/экспорт альбома (с сохранением структуры).
- ✓ Сохранение при выходе.

Рекомендуется использовать следующие сущности:

- User — родитель / опекун
- Photo — фотография (файл, описание, дата)
- Milestone — важное событие (первый шаг, день рождения)
- AgeGroup — возрастная группа
- Timeline — хронология развития

Рекомендуемые паттерны:

- Composite — альбом как коллекция возрастных групп.
- Observer — обновление хронологии при добавлении события.

30) Приложение для изучения скорочтения (1 чел.)

- ✓ Режим показа текста по словам/фразам с заданной скоростью.
- ✓ Настройка скорости (слов в минуту).
- ✓ Тестирование скорости чтения с замером времени.
- ✓ Статистика прогресса: средняя скорость, динамика.
- ✓ Поиск по названию текста, теме, уровню сложности.
- ✓ Фильтрация по жанру (новости, художественное).
- ✓ Импорт/экспорт текстов и статистики.

Рекомендуется использовать следующие сущности:

- User — читатель
- Text — текст (название, содержание, тема, сложность)
- SpeedSetting — настройка скорости (слов/мин)
- TestSession — сессия тестирования
- ProgressTracker — статистика прогресса

Рекомендуемые паттерны:

- Strategy — выбор стратегии отображения (по слову, по фразе).
- Observer — обновление статистики после завершения теста.

3.2 Требования к отчету курсовой работы

Отчет по курсовой работе выполняется каждым студентом в отдельности. Для совместных тем в отчет вносится работа по той части программного продукта, которой занимался студент, следуя общему плану работы.

Оформление. Титульный лист должен содержать: название учебного заведения, наименование кафедры, название дисциплины и тему работы, ФИО и группу учащегося, ФИО и должность руководителя; город и год написания курсовой работы. Текст должен быть набран в текстовом редакторе Microsoft Word или текстовом редакторе свободно распространяемого ПО, сохранен в формате docx или совместимом с docx формате. Основной шрифт – Times New Roman, 14 кегль, межстрочный интервал – одинарный. Каждый абзац с новой красной строки, отступ 1,25 см. Отступы на полях документа: слева 3 см, справа 1 см, верхний и нижний — по 2 см. Выравнивание текста по ширине.

Все рисунки и таблицы должны быть озаглавлены и иметь ссылку на них в тексте. Подписи под рисунками должны располагаться по центру рисунка без точки в конце. Надпись над таблицей выравнивается по левому краю без точки в конце. Листы должны быть пронумерованы, начиная со второй страницы (1 – титульный лист, 2 – оборот титульного листа). Страницы нумеруются снизу, строго по центру. В текст должна быть установлена автоматическая расстановка переносов.

В содержание включают номера и наименования разделов и подразделов с указанием номеров листов (страниц). В содержание также включают все приложения, вошедшие в данный документ, с указанием номера листа (страницы).

Введение. Обзор предметной области по заданной теме в объеме одной страницы. Описать какие существуют приложения по данной и смежной тематике и их возможности, какие видите перспективы по этой теме.

Основная часть. Проектирование интерфейса программного продукта должно соответствовать следующему плану:

1. Анализ задач и пользователей.

1.1. Анализ целевой аудитории:

- общее описание аудитории;
- структура аудитории, выделение сегментов;
- предпочтения целевой аудитории;
- социально-демографические характеристики целевой аудитории;
- тенденции по составу и предпочтениям целевой аудитории на ближайшее будущее.

1.2. Профиль пользователя.

Составить обобщенный профиль представителя целевой аудитории:

- демографические и психографические особенности пользователя;
- задачи и цели пользователя;
- рабочая среда пользователя;
- термины пользователя

1.3. Общие требования и задачи.

2. Выбор репрезентативных задач.

2.1. Список задач пользователя.

2.2. Подробное описание задач в терминах пользователя.

2.3. Список необходимых материалов для выполнения задач.

3. Заимствование.

- Найти и описать 1-2 приложения, в которых реализованы задачи по вашей теме либо похожие задачи.
- Описать существующие низкоуровневые интерфейсные решения, которые можно применить в вашей теме.
- Проанализировать в соответствии с профилем пользователя, какие интерфейсные решения можно заимствовать и выбрать наиболее подходящие для вашей работы;

4. Черновое описание интерфейса.

- Выбрать и обосновать какие модели будут использоваться и для каких частей описания интерфейса.
- Описать интерфейс с помощью выбранных моделей:
 - а) для матрицы прямого манипулирования:
 - описать используемые метафоры;
 - в заданной предметной области определить список объектов, их типы и представления,

- построить матрицу прямого манипулирования объектами;
- б) для модели действий пользователя:
 - определить роли пользователей,
 - составить список действий,
 - построить модель действий пользователя;
- в) для структурной модели:
 - описать используемые элементы интерфейса,
 - составить схемы экранных кадров,
 - графически представить структурную модель;
- г) для последовательно-динамической модели:
 - определить роли пользователей,
 - составить список объектов интерфейса,
 - привести таблицу динамического взаимодействия.

5. Анализ интерфейса.

Проанализировать описанный интерфейс и оптимизировать его на основе полученных результатов.

- 1) Выбрать три главные репрезентативные задачи для исследования.
- 2) Провести CWT анализ выбранных репрезентативных задач и оформить таблицей вида:

Формулировка задачи			
Действие	История	Проблемы	Пути решения

- 3) Провести GOMS анализ выбранных задач.
- 4) Исследовать производительность экранных форм по методу Дж. Раскина касательно выбранных репрезентативных задач.

6. Прототип интерфейса.

- Привести прототипы всех экранных форм, включая сообщения.
- Произвести оценку трех основных экранных форм с помощью законов Фитса и Хика.
- Оптимизировать разработанный вариант интерфейса на основе полученной оценки, а также золотых правил построения интерфейсов.

7. Тестирование с пользователями.

- Описать двоих реальных пользователей либо персонажей в соответствии с профилем пользователя.
- Провести use-case для всех репрезентативных задач для двоих пользователей по форме:

Действие пользователя	Реакция системы	Реакция и ожидания пользователя

- Сделать выводы по возможным улучшениям интерфейса на основании тестирования.

8. Программная реализация.

- Сформулировать основные цели разработки.

- Составить требования к программному продукту.
- Сформировать перечень и характеристики исходных данных.
- Описать получаемые результаты и способы их представления.
- Перечислить использованные классы Qt и их назначение.
- Привести скриншоты основных окон приложения.
- Описать базу данных или структуру сохраняемых файлов.
- Выложить исходный программный код на любой облачный ресурс и привести ссылку в отчете.

Заключение. Сделать выводы о проделанной работе, описать возникшие проблемы в процессе выполнения работы и пути их решения.

Список используемых источников. Перечислить как минимум 3 источника литературы, используемых при подготовке по ГОСТ Р 7.0.5–2008.

3.3 Контрольные вопросы для защиты курсовой работы

1. Перечислите этапы проектирования интерфейса пользователя согласно проблемно-центрированному подходу.
2. Что включает в себя анализ целевой аудитории на этапе проектирования интерфейса, и зачем он необходим?
3. Какова цель этапа заимствования при разработке интерфейса?
4. Какие модели используются для чернового описания интерфейса, и что они отражают в структуре будущего приложения?
5. Что представляет собой SWT-анализ и какую роль он играет при оптимизации пользовательского интерфейса?
6. В чем заключается особенность использования законов Фитса и Хика при оценке пользовательского интерфейса?
7. Что такое GOMS-анализ? Как он применяется для оценки эффективности интерфейса?
8. Какие паттерны проектирования вы использовали в своей работе? Объясните, зачем они нужны и как реализованы в вашем приложении.
9. В чём разница между паттернами Factory и Strategy? Приведите примеры их использования в вашем проекте.
10. Как паттерн Observer помогает в реализации многопользовательского или многокомпонентного интерфейса?
11. Как паттерн Composite может быть полезен при работе с иерархическими структурами (например, альбомы, цели, чек-листы)?
12. Какие данные в вашем приложении сохраняются локально и в каком формате (JSON, XML, SQLite)? Обоснуйте выбор формата.
13. Как реализована поддержка нескольких пользователей в вашем приложении? Какие данные разделяются, а какие — персональны?
14. Как реализованы функции импорта и экспорта данных? Какие ограничения или особенности возникли при их реализации?

СПИСОК ЛИТЕРАТУРЫ

- 1) Qt, Tools for Each Stage of Software Development Lifecycle [Электронный ресурс]. URL: <https://www.qt.io> (дата обращения: 5.05.2025).
- 2) Алексеев Е.Р., Злобин Г.Г., Костюк Д.А. [и др.] Программирование на языке С++ в среде Qt Creator : Курс лекций. — Москва : Интуит НОУ, 2016. — 715 с. [Электронный ресурс]. URL: <https://book.ru/book/918128> (дата обращения: 05.05.2025).
- 3) Шлее М. Qt 5.10. Профессиональное программирование на С++. БХВ-Петербург, 2018. 1052 с.
- 4) Зубкова Т.М. Технология разработки программного обеспечения: учебное пособие. Оренбургский гос. ун-т.- Оренбург: ОГУ, 2017. – 468 с.
- 5) Корнипаев И. Требования для программного обеспечения: рекомендации по сбору и документированию. М.: Издательство «Книга по Требованию», 2013. – 118 с.
- 6) Мерзлякова Е. Ю. Визуальное программирование и человеко-машинное взаимодействие : практикум / Сибирский государственный университет телекоммуникаций и информатики. Новосибирск, 2022. 49 с.